Computing the downward-closure of an indexed language

Richard Mandel
Max Planck Institute for Software Systems, Kaiserslautern

Corto Mascle
Max Planck Institute for Software Systems, Kaiserslautern

Georg Zetzsche
Max Planck Institute for Software Systems, Kaiserslautern

Abstract

We prove the first bounds on the size of a finite automaton recognizing the subword-closure of the language of an indexed grammar. Our upper and lower bounds are both triply exponential for non-deterministic automata, and quadruply exponential for deterministic ones. For the upper bounds, we rely on recent advances in semigroup algebra, which let us compute summaries of bounded size for words with respect to a finite semigroup. By replacing stacks with their summaries, we are able to transform an indexed grammar into a context-free one with the same subword-closure, and then apply existing bounds for context-free grammars.

2012 ACM Subject Classification Theory of computation → Regular languages

Keywords and phrases Higher-order pushdown automata, well quasi-orders, semigroup algebra

1 Introduction

Downward closures For finite words u and v, we say that u is a *(scattered) subword* of v, written $u \preceq v$, if v can be obtained from u by inserting letters. The *downward closure* of a language $L \subseteq \Sigma^*$ is the set $L{\downarrow} = \{u \in \Sigma^* \mid u \preceq v \text{ for some } v \in L\}$ of all subwords of members of L. The downward-closure of a language—its set of all scattered subwords—is a fundamental abstraction that replaces an arbitrary language by a regular one while preserving information about which patterns may occur in its words. Downward-closures play a central role in verification, model checking, and language-theoretic decision procedures, where they provide effective over-approximations of complex infinite behaviours. Examples of applications include verification of thread-pool systems [3], separability problems with respect to piecewise-testable languages [6], analysis of lossy channel networks [1], and verification of asynchronous shared-memory systems [11].

Higman's lemma [9] guarantees that the downward-closure of a language is always regular. This fact makes downward-closures a very natural over-approximation method since we are guaranteed that the resulting languages have a simple (although possibly large) representation. However, turning this existential result into concrete constructions on concrete classes of languages with explicit complexity bounds is a long-standing challenge.

Indexed languages For context-free grammars, tight upper bounds on the size of automata recognising the downward-closure were proven by Courcelle [5]. However, much less was previously understood for indexed languages, the languages recognised by indexed grammars. Those are a powerful extension of context-free grammars introduced by Aho [2] to capture forms of nested dependencies beyond the reach of pushdown automata. Indexed grammars feature unbounded stacks of stack symbols, making even their structural analysis considerably more intricate. Indexed languages can also be defined as the languages recognised by second-order pushdown automata [12], or by Fischer's macro grammars [7]. In 2015, Zetzsche showed

that given an indexed grammar and a finite automaton, one can check whether the language of the latter is the downward-closure of the language of the former [16]. A corollary is that the downward-closure of an indexed language is computable: it suffices to enumerate finite automata until we find one that recognises the downward-closure of the indexed grammar at hand, which happens eventually by Higman's lemma. However, we cannot infer any bounds from this computability proof, and the size of those automata has been an open question since then.

Contribution In this work we establish the first such bounds. Our approach relies on recent advances in finite semigroup theory which provide succinct "summaries" of arbitrarily long words relative to a finite semigroup [8]. By using these summaries to replace indexed stacks, we show how to transform any indexed grammar into a context-free grammar that preserves its downward-closure. Combining this construction with classical bounds for context-free languages yields a triply-exponential upper bound for nondeterministic automata and a quadruply-exponential bound for deterministic automata recognising downward-closures of indexed languages.

We complement these upper bounds with matching lower bounds. For every k, we construct an indexed grammar of size $\mathcal{O}(k^2)$ whose downward-closure requires nondeterministic automata of size at least $2^{2^{2^k}}$ and deterministic ones of size at least $2^{2^{2^{2^k}}}$.

Together, these results provide the first complete quantitative picture of downward-closures in the indexed setting, and highlight the surprising power of semigroup-based techniques for analysing higher-order grammar formalisms.

Structure of the paper

We recall necessary notations and basic results in Section 2. Then in Section 4 we show how to make a given indexed grammar *sound*, meaning that every partial computation can be extended to a full one. In Section 5 we introduce an object at the core of our construction, the *production semigroup*, and then proceed to define *summaries* of stack contents with respect to this monoid. In Section 8 we leverage those summaries to compute from an indexed grammar a context-free one with the same downward-closure, and infer upper-bounds on the size of finite automata recognizing it. Finally, in Section 9 we complement our upper bounds with matching lower bounds.

This paper contains internal links, every occurrence of a term is linked to its definition. *The reader can click on terms (and some notations), or simply hover over them on some pdf readers, to get their definition.*

2 Preliminaries

2.1 Words, trees and languages

Given a finite alphabet Σ, we write Σ^* for the set of words over Σ, and Σ^K for the set of words of length k. Given $w \in \Sigma^*$, we denote $|w|$ its length. We assume familiarity with basic finite automata theory, see [14] for an introduction.

We write $u \preceq v$ if u is a (scattered) *subword* of v; that is, if u can be obtained from v by removing some of its letters. The *downward-closure* of a language $L \subseteq \Sigma^*$ is the set of subwords of words of L, and is denoted $L{\downarrow}$.

▶ **Theorem 2.1** (Consequence of [9], Theorem 4.3)**.** *The downward-closure of a language L is always regular.*

A finite ordered Λ-labelled tree is a pair (τ, λ) with τ a finite prefix-closed subset of $\{0,1\}^*$ such that for all $\nu \in \tau$, $\nu 1 \in \tau$ implies $\nu 0 \in \tau$, and $\lambda : \tau \to \Lambda$ a function mapping each node in τ to a label. We use the usual terminology for trees, with *node, leaf, branch, subtree, parent, child, ancestor, descendant* etc. retaining their usual meaning. The *leaf word* of τ is the word obtained by concatenating $\lambda(\nu_1) \dots \lambda(\nu_k)$, where $\nu_1, \dots, \nu_k$ are the leaves of τ read from left to right, i.e. in lexicographic order.

2.2 Indexed grammars

▶ **Definition 2.2.** *An* indexed grammar *is a tuple of finite sets* (N, T, I, P, S) *with*

- N *the set of non-terminals, and* $S \in N$ *the starting non-terminal*
- T *the set of terminal symbols*
- I *the set of index symbols, also called stack symbols*
- P *a set of productions, which are of the following types:*
  - $A \to Bf$ *with* $A, B \in N$ *and* $f \in I$
  - $Af \to B$ *with* $A, B \in N$ *and* $f \in I$
  - $A \to BC$ *with* $A, B, C \in N$
  - $A \to w$ *with* $w \in T^*$

We define the size *of* $\mathcal{G}$ *as* $|N| + |P| + \sum_{A \to w \in P} |w|$.

A context-free grammar *(or CFG) is an indexed grammar where all productions are of the two last forms.*

A *derivation tree* is a finite ordered tree τ whose nodes are labeled by elements of $NI^* \cup T^*$, with the following constraints. For each node ν with label $A[z]$ (where $A \in N$ and $z \in I^*$), exactly one of the following holds

- ν is a leaf
- ν has two children labeled $B[z]$ and $C[z]$ for some $A \to BC \in P$
- ν has one child labeled $B[fz]$ for some $A \to Bf \in P$
- ν has one child labeled $B[z']$ for some $Af \to B \in P$, with $z = fz'$.

A derivation tree whose root is labeled Az and whose leaf word is equal to u is called a *derivation tree from* $A[z]$ *to* u. If $u \in T^*$ we say that it is a *complete derivation tree*.

A *sentential form* is a word over the (infinite) alphabet $NI^* \cup T$. The set of sentential forms is denoted $\mathbf{SF}_{\mathcal{G}}$, or simply $\mathbf{SF}$ when the grammar is clear from context. An element $A[z] \in NI^*$ is sometimes referred to as a *term*. The derivation relation $\Rightarrow_{\mathcal{G}}$ (or simply $\Rightarrow$) over $\mathbf{SF}$ is defined as:

$$uA[z]v \Rightarrow uB[z]C[z]v \quad \text{if } A \to BC \in P, \qquad uA[z]v \Rightarrow uB[fz]v \quad \text{if } A \to Bf \in P,$$
$$uA[fz]v \Rightarrow uB[z]v \quad \text{if } Af \to B \in P, \qquad uA[z]v \Rightarrow uwv \quad \text{if } A \to w \in P.$$

for all $u, v \in \mathbf{SF}$, $A, B, C \in N$, $f \in I$, $z \in I^*$. We write $\overset{*}{\Rightarrow}_{\mathcal{G}}$ (or simply $\overset{*}{\Rightarrow}$) for the reflexive transitive closure of $\Rightarrow_{\mathcal{G}}$, and the subword relation $\preceq$ is extended to $\mathbf{SF}$ in the natural way, i.e. $u \preceq v$ if u can be obtained from v by deleting terms.

Given a sentential form u, we write $\mathsf{L}_{\mathbf{SF}}(u)$ for the set $\{v \in \mathbf{SF} \mid u \overset{*}{\Rightarrow} v\}$. The *language* of $\mathcal{G}$ is denoted $\mathsf{L}(\mathcal{G})$ and defined as $\mathsf{L}_{\mathbf{SF}}(S) \cap T^*$. Furthermore, for all $X \subseteq N$ and $u \in \mathbf{SF}$, the language $\mathsf{L}_X(u)$ is defined as the set $\mathsf{L}_{\mathbf{SF}}(u) \cap (X \cup T)^*$. In particular, $\mathsf{L}_N(u)$ is the set of sentential forms derivable from u with all stacks empty, and $\mathsf{L}_\emptyset(u)$ is the set of terminal words which can be derived from u. If the language $\mathsf{L}_\emptyset(u)$ is non-empty then we say that u is *productive*.

▶ Remark 2.3. An easy induction shows that for all $A[z] \in NI^*$, a sentential form u can be derived from $A[z]$ if and only if u is the leaf word of a derivation tree whose root is labeled $A[z]$. In the forthcoming proofs we will either use sentential forms or derivation trees depending on what is most convenient.

▶ Remark 2.4. When describing some examples of indexed grammars, we will use rules of the form $Af \to u$ and $A \to u$ with $u \in (N \cup T)^*$. This is just syntactic sugar, as we can replace them with rules from Definition 2.2 while adding a small number of non-terminals, in the spirit of the Chomsky normal form [4]. This transformation induces a linear blow-up in the size of the grammar. Details are left to Appendix A.

▶ **Example 2.5.** We can define the language $L = \{a^n b^{n^2} \mid n \in \mathbb{N}\}$ with an indexed grammar:

$S \to T\ g$ *# g is the stack bottom symbol;*
$T \to T\ f$ *# we push some number n of f;*
$T \to A$
$A\ g \to \varepsilon$ *# if $n = 0$ then return ε;*
$A\ f \to C$ *# if $n > 0$ we pop an f;*
$C \to aAB$ *# repeat previous and current rule n times to get $a^n \prod_{i=0}^{n-1} B\ f^i g$;*
$B\ f \to bbB$
$B\ g \to b$ *#the Bs output $b^{\sum_{i=0}^{n-1}(2i+1)} = b^{n^2}$.*

▶ **Example 2.6.** We can define the language $L = \{w^3 \mid w \in \{a, b\}^*\}$ with an indexed grammar:

$S \to T\ g$ *# g is the stack bottom symbol;*
$T \to T\ f_a$
$T \to T\ f_b$ *# we push some word on the stack, as a sequence of f_a and f_b;*
$T \to WWW$ *# we get three copies of W with the same stack content;*
$W\ g \to \varepsilon$ *# if the stack is empty then return ε;*
$W\ f_a \to aW$
$W\ f_b \to bW$ *# each W outputs the content of the stack.*

▶ Remark 2.7. We can assume without loss of generality that every symbol pushed on the stack carries the information of the production rule used to push it (this property can always be ensured by adding a quantity of new stack symbols and production rules that is at most quadratic in the size of the grammar). Formally, we assume that there are functions $\alpha, \beta : I \to N$ such that for every push rule $A \to Bf$ we have $A = \alpha(f)$ and $B = \beta(f)$.

3 Main results

In this section, we present the main results of this work. Our first main contribution is an algorithm to compute an NFA of at most triply exponential size for the downward closure of an indexed language:

▶ **Theorem 3.1.** *Given an indexed grammar $\mathcal{G}$, one can compute (in triply exponential time) an NFA of at most triply exponential size for $L(\mathcal{G})\downarrow$.*

Moreover, we show that the triply exponential upper bound is tight:

▶ **Theorem 3.2.** *There is a family $(\mathcal{G}_k)_{k\geq 1}$ of indexed grammars of size linear in k such that any NFA for $\mathsf{L}(\mathcal{G}_k)\downarrow$ requires at least $\exp_3(k)$ states.*

Deterministic automata Of course, Theorem 3.1 implies a quadruply exponential upper bound for deterministic automata:

▶ **Corollary 3.3.** *Given an indexed grammar $\mathcal{G}$, one can compute (in quadruply exponential time) a DFA of at most quadruply exponential size for $\mathsf{L}(\mathcal{G})\downarrow$.*

We will also show that the quadruply exponential upper bound is tight:

▶ **Theorem 3.4.** *There is a family $(\mathcal{G}_k)_{k\geq 1}$ of indexed grammars of size linear in k such that any DFA for $\mathsf{L}(\mathcal{G}_k)\downarrow$ requires at least $\exp_4(k)$ states.*

Downward closure comparisons Theorem 3.1 and our technique for Theorem 3.2 allow us to settle the complexity of decision problems related to downward closures of indexed languages. The *downward closure inclusion problem* (for indexed languages) whether two given indexed languages L_1, L_2 satisfy $L_1\downarrow \subseteq L_2\downarrow$. Similarly, the *downward closure equivalence problem* asks whether $L_1\downarrow = L_2\downarrow$.

In [17, Corollary 18], it was shown that downward closure inclusion and equivalence for indexed languages are co-2-NEXP-hard. Here, we settle their precise complexity:

▶ **Theorem 3.5.** *Downward closure inclusion and downward closure equivalence for indexed languages are* co-3-NEXP*-complete.*

4 Soundness

Soundness of an indexed grammar expresses the absence of deadlocks: if we can produce a sentential form u, then from u we can derive a terminal word in T^*.

This property is particularly interesting when it comes to downward-closure computation. In a sound indexed grammar, the following property holds: for all $u \in \mathsf{L}_{\mathbf{SF}}(S)$, if v is such that $v \preceq u$, then $\mathsf{L}_\emptyset(v) \subseteq \mathsf{L}_\emptyset(u)\downarrow$. In other words, we can interleave derivations and deletion of terms without any risk of obtaining "extra" terminal words. This property does not hold in general, because deleting terms that cannot produce terminal words may allow the derivation of a terminal word that is not in the downward closure.

We say that $\mathcal{G}$ is *sound* if for all $u \in \mathsf{L}_{\mathbf{SF}}(S)$ we have $\mathsf{L}_\emptyset(u) \neq \emptyset$, that is, from every sentential form obtained from S we can derive a word in T^*.

▶ **Example 4.1.** The following grammar $\mathcal{G}$ is not sound.

$S \to ABC$

$S \to A$

$A \to Aa$

$A \to \varepsilon$

$B \to Bb$

$B \to \varepsilon$.

In particular, we have $B \preceq ABC \in \mathsf{L}_{\mathbf{SF}}(S)$, but $b^* = \mathsf{L}_\emptyset(B) \not\subseteq \mathsf{L}_\emptyset(ABC)\downarrow = \emptyset$ (since C cannot produce a terminal word). In this case, it is easy to modify $\mathcal{G}$ to obtain a sound grammar which generates the same language.

We will now describe a method to make an indexed grammar sound.

▶ **Definition 4.2.** *For $X \subseteq N$ and $z \in I^*$, we write $z \cdot X$ for the set $\{A \in N \mid \mathsf{L}_X(A[z]) \neq \emptyset\}$. Let $\mathbf{U}$ be defined as the set of non-terminals which can derive a word in T^*, i.e.*

$$\mathbf{U} = \{A \in N \mid \mathsf{L}_\emptyset(A) \neq \emptyset\}.$$

Note that $\mathsf{L}(\mathcal{G})$ is empty if and only if $S \notin \mathbf{U}$. The following lemma states that the operation $z \cdot X$ is a left monoid action of I^* on the set $\mathcal{P}(N)$.

▶ **Lemma 4.3.** *For all $f \in I$, $z \in I^*$ and $X \subseteq N$, we have*

$$fz \cdot X = f \cdot (z \cdot X).$$

Proof. If $A \in f \cdot (z \cdot X)$ then (by definition) $A[f] \overset{*}{\Rightarrow} u$, with $u \in (z \cdot X \cup T)^*$. Let u^z be the sentential form obtained by replacing every non-terminal B in u with $B[Z]$ (i.e. pushing z onto every stack). Since all those non-terminals are in $z \cdot X$, there exists $v \in (X \cup T)^*$ such that $u^z \overset{*}{\Rightarrow} v$, implying that $A[fz] \overset{*}{\Rightarrow} v$ and so $A \in fz \cdot X$.

To show the other inclusion, suppose that $A \in fz \cdot X$ and consider a derivation tree from $A[fz]$ to some $v \in (X \cup T)^*$. Along every branch there is a first node with a label either in T^* or of the form $B[z]$. In the latter case we have $B \in z \cdot X$ (since the tree from this node is a derivation tree from $B[z]$ to an element of $(X \cup T)^*$). After deleting everything below these nodes and removing the z suffix from the stack in each label, we obtain a derivation tree from $A[f]$ to an element of $(z \cdot X \cup T)^*$, completing the proof. ◀

▶ **Lemma 4.4.** *For all $z \in I^*$, $z \cdot \mathbf{U} = z \cdot \emptyset = \{A \in N \mid \mathsf{L}_\emptyset(A[z]) \neq \emptyset\}$.*

Proof. Note that the second equality is simply the definition of $z \cdot \emptyset$. We proceed by induction on z. For $z = \emptyset$, the statement is equivalent to the definition of $\mathbf{U}$. Assuming the statement holds for z, then two applications of Lemma 4.3 yield

$$fz \cdot \mathbf{U} = f \cdot (z \cdot \mathbf{U}) = f \cdot (z \cdot \emptyset) = fz \cdot \emptyset,$$

completing the proof. ◀

The *annotated version* of $\mathcal{G}$, denoted $\overline{\mathcal{G}} = (\overline{N}, T, \overline{I}, \overline{P}, \overline{S})$, is defined as follows. If $S \notin \mathbf{U}$ then it is simply the empty grammar. Otherwise, we have:

- $\overline{N} = \{(A, X) \in N \times 2^N \mid A \in X\}$
- $\overline{I} = I \times 2^N$
- $\overline{P}$ contains the following rules:
  - $(A, X) \to w$ for all $A \to w \in P$ with $w \in T^*$
  - $(A, X) \to (B, X)(C, X)$ for all $A \to BC \in P$ with $A, B, C \in X$
  - $(A, X) \to (B, Y)(f, X)$ for all $A \to Bf \in P$ with $Y = f \cdot X$ and $A \in X$
  - $(A, Y)(f, X) \to (B, X)$ for all $Af \to B \in P$ with $Y = f \cdot X$, $A \in Y$ and $B \in X$
- $\overline{S} = (S, \mathbf{U})$

Finally, we define $\pi : \overline{N}\,\overline{I}^* \cup T \to NI^* \cup T$ to be the morphism projecting each $(A, X) \in \overline{N}$ to A, each $(f, X) \in \overline{I}$ to f and each $a \in T$ to itself.

We now define *annotations*. A nonempty stack word $\overline{z} = (f_n, X_n) \cdots (f_1, X_1) \in \overline{I}^*$ is *consistent* if $X_i = f_i \cdot X_{i-1}$ for all $i > 1$; the empty word is consistent by definition. Note that a consistent stack word $\overline{z}$ is determined by its projection $\pi(\overline{z})$ onto I^* and its last element.

Given $z = f_n \cdots f_1 \in I^*$ and $X \subseteq N$, define $\overline{z}^X$ as $(f_n, X_n) \cdots (f_1, X_1)$ with $X_1 = X$ and $X_{i+1} = f_i \cdot X_i$ for all $i < n$. Given $A[z] \in NI^+$ with $z = f_n \cdots f_1$, an *X-based annotation* of $A[z]$ is a $\overline{\mathcal{G}}$ term $(A, Y)[\overline{z}] = (A, Y)[(f_n, X_n) \cdots (f_1, X_1)] \in \overline{N}\,\overline{I}^*$ such that $X_1 = X$, $Y = f_n \cdot X_n$ and $\overline{z}$ is consistent. In the case of an empty stack, we say that (A, X) is an X-based annotation of A. In particular, by Lemma 4.3 we have $X_i = f_{i-1} \cdots f_1 \cdot X$ for all i and $Y = z \cdot X$. Observe that an X-based annotation exists if and only if $A \in z \cdot X$, and that if it exists it is unique.

▶ **Lemma 4.5.** *$\mathcal{G}$ and $\overline{\mathcal{G}}$ have the same language, and $\overline{\mathcal{G}}$ is sound.*

We prove Lemma 4.5 below, after establishing a preliminary result.

▶ **Lemma 4.6.** *Let $(A, Y)[\overline{z}] \in \overline{N}\,\overline{I}^*$ be an X-based annotation of $A[z] \in NI^*$ for some $X \subseteq N$. If $u \in L_X(A[z])$, then there exists $\overline{u} \in (X \times \{X\} \cup T)^*$ such that*

$$\pi(\overline{u}) = u \text{ and } (A, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\mathcal{G}} \overline{u}.$$

In particular, if $u \in T^*$ then $A[z] \overset{*}{\Rightarrow}_{\mathcal{G}} u$ implies that $(A, X)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u$ as well.

Proof. Let us first assume a nonempty stack, setting $\overline{z} = (f_n, X_n) \cdots (f_1, X_1)$ with $n \geq 1$ and $X = X_1$. We proceed by induction on the length of the derivation $A[z] \overset{*}{\Rightarrow}_{\mathcal{G}} u$, and distinguish cases according to the production rule used in the first step.

- If the rule is of the form $A \to w \in T^*$, then $u = w$ and the statement is immediate.
- If the rule is of the form $Af \to B$, then f is necessarily equal to f_n, and we have $z = f_n z'$ with $Bz' \overset{*}{\Rightarrow}_{\mathcal{G}} u$. Let $\overline{z}'$ be such that $\overline{z} = (f_n, X_n)\overline{z}'$. Since $(A, Y)[\overline{z}]$ is an X-based annotation of $A[z]$, we must have $X_n = z' \cdot X$, and since $B[z']$ derives a word in $(X \cup T)^*$ it follows that $B \in X_n$. Hence, there is a corresponding derivation $(A, Y)[(f_n, X_n)\overline{z}'] \Rightarrow_{\overline{\mathcal{G}}} (B, X_n)[\overline{z}']$. Moreover, $(B, X_n)[\overline{z'}]$ is an X-based annotation of $B[z']$, so the induction hypothesis yields the derivation $(B, X_n)[\overline{z'}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}$ for some $\overline{u} \in (X \times \{X\} \cup T)^*$ with $\pi(\overline{u}) = u$. Hence, $(A, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}$ as desired. Note that this establishes the base case of the induction, since a derivation of length one must consist of either this rule or the previous one (and if $u = B$ then we simply set $\overline{u} = (B, X)$).
- If the rule is of the form $A \to BC$, then $u = u_B u_C$ where $B[z] \overset{*}{\Rightarrow}_{\mathcal{G}} u_B$ and $C[z] \overset{*}{\Rightarrow}_{\mathcal{G}} u_C$. By our assumption, we have $Y = z \cdot X$, and since $A[z]$, $B[z]$ and $C[z]$ can all produce a word in $(X \cup T)^*$, we must have $A, B, C \in Y$ (also, $A \in Y$ by definition). Consequently, we may apply the corresponding derivation in $\overline{\mathcal{G}}$: $(A, Y)[\overline{z}] \Rightarrow_{\overline{\mathcal{G}}} (B, Y)[\overline{z}](C, Y)[\overline{z}]$. Note that $(B, Y)[\overline{z}]$ and $(C, Y)[\overline{z}]$ are X-based annotations of $B[z]$ and $C[z]$ respectively, so by the induction hypothesis there are $\overline{u}_B, \overline{u}_C \in (X \times \{X\} \cup T)^*$ such that $\pi(\overline{u}_B) = u_B$, $\pi(\overline{u}_C) = u_C$, $(B, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}_B$ and $(C, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}_C$. Setting $\overline{u} = \overline{u}_B \overline{u}_C$, we thus have $\pi(\overline{u}) = u$ and $(A, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}$, as desired.
- If the rule is of the form $A \to Bf$, then $B[fz] \overset{*}{\Rightarrow}_{\mathcal{G}} u$. As before, we have $Y = z \cdot X$, and $A \in Y$ by definition. Let $Y' = f \cdot Y = fz \cdot X$. Since $B[fz]$ can produce a word in $(X \cup T)^*$, it must be the case that $B \in Y'$. Hence, we have the corresponding derivation $(A, Y)[\overline{z}] \Rightarrow_{\overline{\mathcal{G}}} (B, Y')[(f, Y)\overline{z}]$. Note that $(B, Y')[(f, X)\overline{z}]$ is an X-based annotation of $B[fz]$, so from the induction hypothesis we obtain $\overline{u} \in (X \times \{X\} \cup T)^*$ such that $\pi(\overline{u}) = u$ and $(B, Y')[(f, Y)\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}$. Hence, $(A, Y)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} \overline{u}$.

This concludes the proof for a nonempty stack. In the case where z is empty, observe that only the first, second and fourth rules may be the first step in a derivation, and in those cases the arguments all go through unchanged (i.e. with $Y = X$). ◀

Proof of Lemma 4.5. The inclusion $\mathsf{L}(\overline{\mathcal{G}}) \subseteq \mathsf{L}(\mathcal{G})$ is obtained as follows. For all $w \in \mathsf{L}(\overline{\mathcal{G}})$, we have a derivation tree for $\overline{\mathcal{G}}$ from $(S, \mathbf{U})$ to w. Since π maps the rules in $\overline{P}$ to rules in P, it follows that the tree obtained by applying π to each node is a derivation tree from S to w for $\mathcal{G}$, implying that $w \in \mathsf{L}(\mathcal{G})$.

To obtain $\mathsf{L}(\mathcal{G}) \subseteq \mathsf{L}(\overline{\mathcal{G}})$, suppose that $w \in \mathsf{L}(\mathcal{G})$. Then there is a derivation $S \overset{*}{\Rightarrow}_{\mathcal{G}} w$, and since $(S, \mathbf{U})$ is a $\mathbf{U}$-based annotation of S it follows from Lemma 4.6 that there is a derivation$(S, \mathbf{U}) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$, implying that $w \in \mathsf{L}(\overline{\mathcal{G}})$.

It remains to prove soundness. Given $\overline{u} \in (\overline{N}\,\overline{I}^* \cup T)^*$, we say that $\overline{u}$ is *sound* if every term $(A, X)[\overline{z}]$ of $\overline{u}$ is a $\mathbf{U}$-based annotation of some $A[z] \in NI^*$.

We wish to show that

1. every $\overline{u}$ such that $(S, \mathbf{U}) \Rightarrow_{\overline{\mathcal{G}}} \overline{u}$ is sound;
2. for every sound $\overline{u}$ there exists $w \in T^*$ such that $u \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$.

The first point follows from the definition of $\overline{\mathcal{G}}$ by an easy induction on the derivation $(S, \mathbf{U}) \Rightarrow_{\overline{\mathcal{G}}} \overline{u}$. For the second point, it is enough to prove it for a single term $\overline{u} = (A, X)[\overline{z}]$, since a sentential form can produce a terminal word if and only if all its terms can.

As $(A, X)[\overline{z}]$ is a $\mathbf{U}$-based annotation of some $A[z]$, then $X = z \cdot \mathbf{U}$. Since $A \in X$ by definition of $\overline{N}$, there is a derivation from $A[z]$ to a word of T^*. As a result, by Lemma 4.6 there is a derivation from $(A, X)[\overline{z}]$ to a word in T^*. ◀

▶ Remark 4.7. Although we are able to turn any indexed grammar into a sound one with the same language, this transformation incurs an exponential blow-up in the size. Therefore, in order to obtain tight complexity bounds, we do not simply assume soundness of the input grammar. Rather, we work with annotated versions explicitly when needed.

5 The production semigroup

Another key aspect of our construction is the production semigroup. It is a finite semigroup in which we evaluate stack contents. Essentially, we map a stack content $z \in \overline{I}^*$ to a semigroup element which says, for each pair A, B of non-terminals, whether we can obtain B from $A[z]$.

The idea is that since we only want to compute the language of $\mathcal{G}$ *up to downward-closure*, if we have a derivation from $A[z]$ to a sentential form uBv, we can erase u and v, thus turning $A[z]$ into B. Even better, if we have a derivation from $A[z]$ to A then, roughly speaking, this means we can turn any term $A[zz']$ into $A[z']$. We aim to find such patterns in order to transmit the flexibility given by the subword-closure to the stack contents.

Let us now start by describing formally the semigroup at hand. It is a bit more involved than what is described above since we have to account for the soundness issues explained in the previous part.

We first introduce some terminology. A semigroup $(\mathbf{S}, \cdot)$ is a set equipped with an associative product. We will often identify a semigroup with its set of elements and denote it simply as $\mathbf{S}$, when the product operation is clear. A monoid $(\mathbf{M}, \cdot, \mathbf{1_M})$ is a semigroup with a neutral element $\mathbf{1_M}$.

Given a semigroup $(\mathbf{S}, \cdot)$, define $\psi_{\mathbf{S}} : \mathbf{S}^* \to \mathbf{S}$ to be its *evaluation morphism*, which maps a sequence of elements of $\mathbf{S}$ to its product. An element $x \in \mathbf{S}$ is *idempotent* if $x \cdot x = x$. The set of idempotent elements of $\mathbf{S}$ is denoted $\mathbf{Idem}(\mathbf{S})$.

In all that follows we fix an indexed grammar $\mathcal{G} = (N, T, I, P, S)$. We will mainly work with its annotated version $\overline{\mathcal{G}}$. Note that we cannot use the annotation construction as a black box and simply assume that the input grammar is sound (see Remark 4.7).

Define $\mathbb{M}$ as the boolean matrix monoid over non-terminals of $\mathcal{G}$: An element of $\mathbb{M}$ is a matrix in $\mathbb{B}^{N\times N}$, with $\mathbb{B} = \{\mathbf{true}, \mathbf{false}\}$. The product is the usual product of matrices over the boolean ring $(\mathbb{B}, \vee, \wedge)$, and the neutral element is the matrix with **true** on the diagonal and **false** everywhere else. Given matrices M_1, M_2, we write M_1M_2 for their product.

We also define, for all $X \subseteq N$, the *reachability relation* $\mathcal{R}_X$ over N. It relates two non-terminals if from the first one we can produce a sentential form containing the second one, with empty stacks. Formally,

$$A \ \mathcal{R}_X \ B \text{ if and only if there is a derivation } (A, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(B, X)v \text{ with } u, v \in \mathbf{SF}$$

(note that due to the soundness property, it is only important that (B, X) have an empty stack).

Let α, β be the functions described in Remark 2.7 for $\mathcal{G}$.

Define $\mathbb{S}$ as the following semigroup. An element of $\mathbb{S}$ is either a tuple (B, Y, M, A, X) with $A, B \in N$ non-terminals, $X, Y \subseteq N$ sets of non-terminals, and $M \in \mathbb{M}$, or $\bot$. The product operation is defined as follows:

$$(B_2, Y_2, M_2, A_2, X_2)\cdot(B_1, Y_1, M_1, A_1, X_1) = \begin{cases} (B_2, Y_2, M_1M_2, A_1, X_1) \text{ if } X_2 = Y_1 \text{ and } B_1 \ \mathcal{R}_{X_2} \ A_2 \\ \bot \text{ otherwise.} \end{cases}$$

Further, for all (B, Y, M, A, X) we set $\bot\cdot(B, Y, M, A, X) = (B, Y, M, A, X)\cdot\bot = \bot\cdot\bot = \bot$.

We define a morphism $\varphi : \overline{I}^+ \to \mathbb{S}$ as follows. For each letter $(f, X) \in I$, set

$$\varphi(f, X) = (\beta(f), f \cdot X, M_{f,X}, \alpha(f), X),$$

where for all $A, B \in N$, $M_{f,X}(A, B) = \top$ if and only if there exist $u, v \in (X \cup T)^*$ such that $A[f] \overset{*}{\Rightarrow}_{\mathcal{G}} uBv$.

We say that a non-empty stack $\overline{z} = (f_n, X_n)\cdots(f_1, X_1) \in \overline{I}^*$ is *feasible* if there is a derivation $(\alpha(f_1), X_1) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(\beta(f_n), f_n \cdot X_n)[\overline{z}]v$ with $u, v \in \mathbf{SF}$.

▶ **Lemma 5.1.** *Let $\overline{z} = (f_n, X_n)\cdots(f_1, X_1) \in \overline{I}^*$ be a non-empty stack. The following are equivalent:*

1. *$\overline{z}$ is feasible*
2. *$\varphi(\overline{z}) \neq \bot$*
3. *for all $i > 1$, $X_i = f_i \cdot X_{i-1}$, $\beta(f_{i-1}) \ \mathcal{R}_{X_i} \ \alpha(f_i)$.*

The proof is given in Appendix B

Let $(A, X) \in \overline{N}$, we say that $(A, X)\overline{z}$ is *feasible* if $\overline{z}$ is feasible, $X = f_n \cdot X_n$ and $\beta(f_n) \ \mathcal{R}_X \ A$.

The following two lemmas relate $\mathbb{M}$ and $\mathbb{S}$ to derivations in $\mathcal{G}, \overline{\mathcal{G}}$.

▶ **Lemma 5.2.** *Let $z \in I^*$, $X \subseteq N$ with $|z| \geq 1$ and let $(B, Y, M, A, X) = \varphi(\overline{z}^X)$, we have, for all $C \in X$ and $D \in Y$,*

$$C \ \mathcal{R}_X \ A \text{ and } B \ \mathcal{R}_Y \ D$$
$$\Updownarrow$$
$$\text{there exist } u, v \in \mathbf{SF} \text{ such that } (C, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(D, Y)[\overline{z}^X]v.$$

This lemma is proven in Appendix C.

▶ **Lemma 5.3.** *Let $z \in I^*$, $X \subseteq N$ with $|z| \geq 1$ and let $(B, Y, M, A, X) = \varphi(\overline{z}^X)$, we have*

$$M(D, C) = \top$$
$$\Updownarrow$$
$$C \in X,\ D \in Y \text{ and there exist } u, v \in (X \cup T)^* \text{ such that } D[z] \overset{*}{\Rightarrow}_{\mathcal{G}} uCv.$$

This lemma is proven in Appendix D.

6 Pumping and skipping

The *pump rule* lets us push an arbitrary sequence mapping to an idempotent e. Formally, it is defined as follows:

$$(A, X)[z_e \overline{z}] \to_{pump} (A, X)[z'_e z_e \overline{z}]$$

for all $(A, X) \in \overline{N}$, $z_e, z'_e, \overline{z} \in \overline{I}$ and $e \in \mathbf{Idem}(\mathbb{S})$ such that $\varphi(z_e) = \varphi(z'_e) = e$.

The *skip rule* is defined as follows:

$$(A, X)[z_e v_1 \dots v_N \overline{z}] \to_{skip} (A, X)[v_1 \dots v_N \overline{z}]$$

for all $(A, X) \in \overline{N}$, $u_1, \dots, u_N, z_e, \overline{z} \in \overline{I}$ and $e \in \mathbf{Idem}(\mathbb{S})$ such that $\varphi(u_1) = \dots = \varphi(u_N) = \varphi(z_e) = e$.

We now show that these additional rule do not increase the downward-closure of the language of our indexed grammar.

▶ **Lemma 6.1.** *Let $M \in \mathbb{M}$ a matrix. If $MM = M$ and we have terms $A_1, \dots, A_{N+1} \in N$ such that $M(A_i, A_{i+1}) = \top$ for all i then there exists i such that $M(A_i, A_i) = \top$.*

Proof. By pigeonhole principle, there exist $i < j$ such that $A_i = A_j$. Since M is idempotent, $M^{j-i} = M$. Since $M(A_i, A_{i+1}) = \dots = M(A_{j-1}, A_j) = \top$, we have $M(A_i, A_j) = \top$. Hence $M(A_i, A_i) = \top$. ◀

▶ **Proposition 6.2.** $L^{pump,skip}(\overline{\mathcal{G}}) \subseteq L(\overline{\mathcal{G}})\downarrow$

The proof is presented in Appendix H.

7 Summarising stack contents

In the previous Sections we have shown that adding the pump rule and skip rule to our annotated indexed grammar does not change its downward-closure. Those rules give us a lot of flexibility on the infixes of the stack mapping to idempotents.

Specifically, if we have a sequence of contiguous infixes which all map to the same idempotent e, we can replace it with any other, as long as the last N infixes are preserved. We will now see how to compress words to bounded summaries, where such sequences of infixes are abstracted away, by remembering only the N last of them. To do so, we rely on a recent construction presented in [8]. There, the authors show such a method to compress words with respect to a boolean matrix monoid to summaries of exponential size in the dimension of the matrices. This construction is in the spirit of the well-known Simon's factorisation forest theorem [15].

We do a self-contained proof in this paper, since our notations are quite different and our summaries need to be constructed slightly differently: they need to remember only the first and last infixes while we need only the N last ones, and they construct their summaries as

trees of bounded height, bottom-up, while we need to build ours from right to left along the stack content.

We recall some notions of semigroup algebra which will be necessary in the next section.

Let $(\mathbf{S}, \cdot)$ be a finite semigroup. Let $(\mathbf{S}^1, \cdot)$ be the monoid obtained by extending $\mathbf{S}$ with a neutral element. Formally, $\mathbf{S}^1 = \mathbf{S} \cup \{\mathbf{1}\}$ with $\mathbf{1} \cdot x = x \cdot \mathbf{1} = x$ for all $x \in \mathbf{S}^1$.

We define the usual Green relations $\mathcal{J}, \mathcal{L}, \mathcal{R}, \mathcal{H}$ on $\mathbf{S}$, starting with the following partial orders:

- $x \leq_{\mathcal{J}} y$ if there exist $a, b \in \mathbf{S}^1$ such that $x = a \cdot y \cdot b$
- $x \leq_{\mathcal{L}} y$ if there exist $a \in \mathbf{S}^1$ such that $x = a \cdot y$
- $x \leq_{\mathcal{R}} y$ if there exist $b \in \mathbf{S}^1$ such that $x = y \cdot b$
- $x \leq_{\mathcal{H}} y$ if $x \leq_{\mathcal{L}} y$ and $x \leq_{\mathcal{R}} y$

The relations $\mathcal{J}, \mathcal{L}, \mathcal{R}, \mathcal{H}$ are the equivalence relations induced by those partial orders: for each $\mathcal{X} \in \{\mathcal{J}, \mathcal{L}, \mathcal{R}, \mathcal{H}\}$ we define $\mathcal{X} = \leq_{\mathcal{X}} \cap \geq_{\mathcal{X}}$, as well as $<_{\mathcal{X}} = \leq_{\mathcal{X}} \setminus \geq_{\mathcal{X}}$. We do not include the semigroup $\mathbf{S}$ in the notation since it will always be clear from the context.

The *regular $\mathcal{J}$-length* [1] of $\mathbf{S}$, denoted $JL(\mathbf{S})$, is defined as

$$JL(\mathbf{S}) = \sup\{k \in \mathbb{N} \mid \exists e_1 >_{\mathcal{J}} \cdots >_{\mathcal{J}} e_k \in \mathbf{Idem}(\mathbf{S})\}.$$

▶ **Definition 7.1.** *The* regular $\mathcal{J}$-height *of an element $x \in \mathbf{S}$ is the maximal number h such that there exist idempotents $e_1, \ldots, e_h \in E(\mathbf{S})$ such that $e_1 >_{\mathcal{J}} \cdots >_{\mathcal{J}} e_k \geq_{\mathcal{J}} x$.*

We write $\mathbf{height}(x)$ *for the regular $\mathcal{J}$-height of an element $x \in \mathbf{S}$. We extend this notation to sequences of elements: Given a sequence of elements $u \in \mathbf{S}^*$, we write* $\mathbf{height}(u)$ *instead of* $\mathbf{height}(\psi_{\mathbf{S}}(u))$.

▶ Remark 7.2. Observe that $\mathbf{height}(x)$ is in $[1, JL(\mathbf{S})]$ for all $x \in \mathbf{S}$. The upper bound follows from the definition of $JL(\mathbf{S})$. The fact that $\mathbf{height}(x)$ is always positive follows from the fact that in a finite semigroup, for all element x there exists n such that x^n is idempotent, hence there is an idempotent $\leq_{\mathcal{J}}$-below x.

In what follows we will extend $\overline{I}$ with additional letters, one fresh letter e^+ for each idempotent $e \in \mathbf{Idem}(\mathbb{S})$. We set $\varphi(e^+) = e$ for all such e. Intuitively, e^+ represents a large sequence of factors, which all evaluate to e.

We extend the regular $\mathcal{J}$-height function naturally: Given a sequence of stack symbols $z \in \overline{I}^*$, we write $\mathbf{height}(z)$ instead of $\mathbf{height}(\varphi(z))$. Similarly, the regular $\mathcal{J}$-height of a sequence of words $z_1 \ldots z_n \in (\overline{I}^*)^*$ is the height of their concatenation, and so on for sequences of sequences of sequences...

▶ **Definition 7.3.** *A 0-summary is simply the empty word ε. Let $h \in [1, JL(\mathbb{S})]$.*

- *An h-*atom *is a word $x\sigma$ with $x \in \mathbb{S}$ and σ an h'-summary for some $h' < h$, so that* $\mathbf{height}(x\sigma) = h$.
- *An h-*block *is a sequence of the form $e^+v_1 \cdots v_N w$ with $v_1, \ldots, v_N$ and w sequences of h-atoms and $e \in \mathbf{Idem}(\mathbb{S})$ such that $\varphi(v_i) = e$ for all i and* $\mathbf{height}(e^+v_1 \cdots v_N w) = h$.
- *An h-*summary *is a sequence of the form $\sigma u B_1 \ldots B_k$ with σ an $(h-1)$-summary, u a word of h-atoms and $B_1, \ldots, B_k$ h-blocks, so that* $\mathbf{height}(\sigma u B_1 \ldots B_k) = h$.

A summary *is an h-summary for some h. The set of summaries is denoted* **Summaries**.

[1] Called regular $\mathcal{D}$-length in [10]. For finite semigroups, $\mathcal{D} = \mathcal{J}$, and we use $\mathcal{J}$ here since it is more common.

We define the following operation on summaries: Given $h \in [1, JL(\mathbb{S})]$, an element $x \in \mathbb{S}$ and an h-summary $\sigma = \sigma' u B_1 \dots B_k$, we define $\texttt{push}(x \blacktriangleright \sigma)$ as follows:

1. If $\mathbf{height}(x\sigma) > h$ then let $h_+ = \mathbf{height}(x\sigma)$; $\texttt{push}(x \blacktriangleright \sigma)$ is the h_+-summary made of a single h_+-atom $x\sigma$
2. Otherwise we have $\mathbf{height}(x\sigma) \leq h$. Since $\mathbf{height}(x\sigma) \geq \mathbf{height}(\sigma) = h$, we even have $\mathbf{height}(x\sigma) = h$.
   a. If $\mathbf{height}(x\sigma') < h$ then $\texttt{push}(x \blacktriangleright \sigma) = (\texttt{push}(x \blacktriangleright \sigma'))uB_1 \dots B_k$
   b. Otherwise, we have $\mathbf{height}(x\sigma') = h$ (since $\mathbf{height}(x\sigma') \leq \mathbf{height}(x\sigma) \leq h$), hence $x\sigma'$ is an h-atom.
      i. If the sequence of h-atoms $(x\sigma')u$ is of the form $v_0 \dots v_N w$ with $\varphi(v_i) = e$ for all i, for some $e \in \mathbf{Idem}(\mathbb{S})$, then define the h-block $B = e^+ v_1 \dots v_N w$ (note that e^+ replaces the first factor v_0).
         A. If there exists j such that B_j is of the form $e^+ v'_1 \dots v'_N w'$ and moreover we have $\varphi(v_1 \dots v_N B_1 \dots B_{j-1} u'_1 \dots u'_N e) = e$ then we set $\texttt{push}(x \blacktriangleright \sigma) = \sigma' B_j B_{j+1} \dots B_k$. If there are several such j we pick the maximal one.
         B. Otherwise $\texttt{push}(x \blacktriangleright \sigma) = BB_1 \dots B_k$
      ii. Otherwise $\texttt{push}(x \blacktriangleright \sigma) = u' B_1 \dots B_k$ with $u' = (x\sigma')u$

We also define the inverse relation: for all $x \in \mathbb{S}$, σ, σ' summaries, $\sigma \in \texttt{pop}(x \blacktriangleleft \sigma')$ if and only if $\sigma' = \texttt{push}(x \blacktriangleright \sigma)$. Given $\overline{z} \in \overline{I}^*$ we write $\texttt{push}(\overline{z} \blacktriangleright \sigma)$ and $\texttt{pop}(\overline{z} \blacktriangleleft \sigma)$ instead of $\texttt{push}(\varphi(\overline{z}) \blacktriangleright \sigma)$ and $\texttt{pop}(\varphi(\overline{z}) \blacktriangleleft \sigma)$.

The *size* of an h-summary (resp. h-atom, h-block) is defined recursively naturally as the sum of the sizes of its elements, the size of a single letter being 1. Although summaries can have a priori unbounded sizes, we can show that the sizes of the ones we can actually obtain by pushing a sequence of letters is exponentially bounded in N. To prove this, we rely on two results of Jecker [10], recalled in Appendix E:

- One guarantees that in a word of elements of exponential length in $JL(\mathbb{S})$ we can find large sequences of consecutive factors that all map to the same idempotent (Theorem E.3).
- Another one bounds the regular $\mathcal{J}$-length of a boolean matrix monoid by a polynomial in its dimension (see Theorem E.4).

We can prove that, when considering $(h-1)$-summaries as single letters, we can bound the size of a h-summary by an exponential in N. The bound on summaries results from the product of those (polynomially many) exponential functions. This proof is heavily inspired by the proof of Theorem 26 in [8].

▶ **Lemma 7.4.** *For all $z \in \overline{I}^*$, $\texttt{push}(z \blacktriangleright \varepsilon)$ is of size at most exponential in N.*

This lemma is proven in Appendix E.

8 A CFG for the downward-closure of the indexed grammar

8.1 Definition of the grammar and first inclusion

We define a context-free grammar $\mathcal{C}_\mathcal{G}$ which over-approximates $\overline{\mathcal{G}}$ by replacing the stack contents with their summaries in $\mathbb{S}$. We will see that while the language of $\mathcal{C}_\mathcal{G}$ is in general larger than the one of $\overline{\mathcal{G}}$, their downward-closures are the same.

To begin with, let us restrict the set of summaries to the ones that can actually result from a derivation of the indexed grammar.

A triple (A, X, σ) is *feasible* if $\mathtt{push}(\overline{z} \blacktriangleright \varepsilon) = \sigma$ for some feasible $\overline{z}$. The set of feasible triples is denoted $\mathbf{FT}$.

Define the following context-free grammar $\mathcal{C}_\mathcal{G}$: Its set of non-terminals is $\mathbf{FT}$, with $(S, \mathbf{U}, \varepsilon)$ the initial one. The set of terminal symbols is T, and the production rules are as follows:

- If $(A, X) \to (B, Y)(C, Z) \in \overline{P}$ then $(A, X, \sigma) \to (B, Y, \sigma)(C, Z, \sigma)$, for all $\sigma \in \mathbf{FT}$.
- If $(A, X) \to (B, Y)(f, X) \in \overline{P}$ then $(A, X, \sigma) \to (B, Y, \mathtt{push}((f, X) \blacktriangleright \sigma))$, for all σ such that $(A, X, \sigma) \in \mathbf{FT}$ (which implies $(B, Y, \mathtt{push}((f, X) \blacktriangleright \sigma)) \in \mathbf{FT}$, as is easily verified).
- If $(A, X)(f, X) \to (B, Y) \in \overline{P}$ then $(A, X, \sigma) \to (B, Y, \sigma')$, for all $\sigma \in \mathbf{FT}$ and $\sigma' \in \mathtt{pop}((f, X) \blacktriangleleft \sigma)$ such that $(B, Y, \sigma') \in \mathbf{FT}$.

Our main theorem is that the downward-closures of the languages of the indexed grammar and this context-free grammar are the same.

▶ **Theorem 8.1.** *The language of $\mathcal{C}_\mathcal{G}$ has the same downward-closure as $L(\mathcal{G})$.*

One of the directions is quite easy: we can simply show that the language of $\mathcal{C}_\mathcal{G}$ contains the one of $\mathcal{G}$. This is natural as $\mathcal{C}_\mathcal{G}$ is built as an over-approximation of $\mathcal{G}$.

▶ **Proposition 8.2.** $L(\overline{\mathcal{G}}) \subseteq L(\mathcal{C}_\mathcal{G})$.

Proof. Let $(A, X)[z]$ be a feasible term and $w \in T^*$ and $(A, X)[z] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$. We show that $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$, by induction on the derivation. Note that since $(A, X)z$ is feasible, $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon))$ is indeed a non-terminal of $\mathcal{C}_\mathcal{G}$. We separate cases according to the first rule applied in the derivation $(A, X)[z] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$.

- If it is of the form $(A, X) \to w \in T^*$ then we have $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \to w$ in $\mathcal{C}_\mathcal{G}$, hence $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$.
- If it is of the form $(A, X) \to (B, X)(C, X)$ then we have $w_B, w_C \in T^*$ such that $w = w_B w_C$ and $(B, X)[z] \overset{*}{\Rightarrow}_\mathcal{G} w_B$ and $(C, X)[z] \overset{*}{\Rightarrow}_\mathcal{G} w_C$.
  By induction hypothesis, $(B, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w_B$ and $(C, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w_C$.
  Further, we have $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \to (B, X, \mathtt{push}(z \blacktriangleright \varepsilon))(C, X, \mathtt{push}(z \blacktriangleright \varepsilon))$ in $\mathcal{C}_\mathcal{G}$, hence $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \Rightarrow_{\mathcal{C}_\mathcal{G}} (B, X, \mathtt{push}(z \blacktriangleright \varepsilon))(C, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w_B w_C = w$.
- If it is of the form $(A, X) \to (B, Y)(f, X)$ then we have $(B, Y)[(f, X)z] \overset{*}{\Rightarrow}_\mathcal{G} w$. Since $(A, X)[z]$ is feasible, so is $(B, Y)[(f, X)z]$.
  By induction hypothesis, $(B, Y, \mathtt{push}((f, X)z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$. Furthermore, we have $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \to (B, Y, \mathtt{push}((f, X) \blacktriangleright \mathtt{push}(z \blacktriangleright \varepsilon)))$ in $\mathcal{C}_\mathcal{G}$, and $\mathtt{push}((f, X) \blacktriangleright \mathtt{push}(z \blacktriangleright \varepsilon)) = \mathtt{push}((f, X)z \blacktriangleright \varepsilon)$ by definition.
  Therefore, $(A, X, \mathtt{push}(z \blacktriangleright \varepsilon)) \Rightarrow_{\mathcal{C}_\mathcal{G}} (B, Y, \mathtt{push}((f, X)z \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$.
- If it is of the form $(A, Y)(f, X) \to (B, X)$ then we have $(B, X)z' \overset{*}{\Rightarrow}_\mathcal{G} w$ with z' such that $z = (f, X)z'$. Since $(A, Y)[(f, X)z']$ is feasible, so is $(B, X)[z']$. By induction hypothesis, $(B, X, \mathtt{push}(z' \blacktriangleright \varepsilon)) \Rightarrow_{\mathcal{C}_\mathcal{G}} w$.
  Furthermore, we have $\mathtt{push}(z \blacktriangleright \varepsilon) = \mathtt{push}((f, X) \blacktriangleright \mathtt{push}(z' \blacktriangleright \varepsilon))$ by definition and therefore $\mathtt{push}(z' \blacktriangleright \varepsilon) \in \mathtt{pop}((f, X) \blacktriangleleft \mathtt{push}(z \blacktriangleright \varepsilon))$.
  As a consequence, $(A, Y, \mathtt{push}(z \blacktriangleright \varepsilon)) \to (B, X, \mathtt{push}(z' \blacktriangleright \varepsilon))$ in $\mathcal{C}_\mathcal{G}$, and therefore $(A, Y, \mathtt{push}(z \blacktriangleright \varepsilon)) \Rightarrow_{\mathcal{C}_\mathcal{G}} (B, X, \mathtt{push}(z' \blacktriangleright \varepsilon)) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$.

It suffices to apply this property to $(S, \mathbf{U})$: for all $w \in T^*$, if $w \in L(\mathcal{G})$ then $(S, \mathbf{U}) \overset{*}{\Rightarrow}_\mathcal{G} w$, which implies $(S, \mathbf{U}, \varepsilon) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w$, and thus $w \in L(\mathcal{C}_\mathcal{G})$. ◀

8.2 Simulating $\mathcal{C}_\mathcal{G}$ with pumps and skips

For the other direction, we will simulate an execution of $\mathcal{C}_\mathcal{G}$ by replacing summaries with actual stacks, where symbols e^+ are replaced with a sequence of many factors of stack symbols evaluating to e. We will use the pump rule and skip rule defined above to extend and reduce sequences of factors mapping to the same idempotent. This will allow us to simulate derivations of $\mathcal{C}_\mathcal{G}$ with $\overline{\mathcal{G}}$ by adjusting the factors corresponding to symbols e^+.

We will now show that every word resulting from a derivation of $\mathcal{C}_\mathcal{G}$ is a subword of a word obtained by a derivation with pump and skip in $\overline{\mathcal{G}}$. We simply mimic a derivation of $\mathcal{C}_\mathcal{G}$ by applying the same rules, with some pump rules and skip rules to adjust the sequences of idempotents corresponding to letters e^+ in $\mathcal{C}_\mathcal{G}$.

▶ **Definition 8.3.** *Let $K \in \mathbb{N}$, $z \in \overline{\Gamma}^*$, we say that z is an* expansion *of an h-atom/ h-block / h-summary if the following conditions are satisfied. Those conditions are defined inductively on h. For $h = 0$, we say that z is an expansion of a 0-summary (i.e., ε) if $z = \varepsilon$.*

- *z is an expansion of an h-atom $a\sigma'$ if $z = az'$ with z' an expansion of σ'.*
- *z is an expansion of e^+ if $z = z_1 \cdots z_p$ with $\varphi(z_i) = e$ for all i and for all realisation z' of e such that $|z'| \leq K$, there exists i such that $z_i = z'$.*
- *z is an expansion of a sequence of h-atoms $\alpha_1 \cdots \alpha_m$ if $z = z_1 \cdots z_m$ with z_i an expansion of α_i for all i.*
- *z is an expansion of an h-block $e^+ v_1 \dots v_N w$ if $z = z_e z^v z^w$ with z^v (resp. z^w, z_e) an expansion of $v_1 \cdots v_N$ (resp. w, e^+).*
- *z is an expansion of an h-summary $\sigma' u B_1 \dots B_m$ if $z = z^{\sigma'} z^u z_1 \dots z_m$ with $z^{\sigma'}$ an expansion of σ', z^u an expansion of u and z_i an expansion of B_i for all i.*

▶ **Lemma 8.4.** *Let $(A, X, \sigma_1) \to (B, Y, \sigma_2)$ be a rule of $\mathcal{C}_\mathcal{G}$ with $\sigma_2 = \texttt{push}((f, X) \blacktriangleright \sigma_1)$. Let $\overline{z}_1$ an expansion of σ_1. Then $(f, X)\overline{z}_2$ is an expansion of σ_2.*

This lemma is proven in Appendix F.

▶ **Lemma 8.5.** *Let $(A, X, \sigma_1) \to (B, Y, \sigma_2)$ be a rule of $\mathcal{C}_\mathcal{G}$ with $\sigma_2 \in \texttt{pop}((f, Y) \blacktriangleleft \sigma_1)$. Let $\overline{z}_1$ an expansion of σ_1. Then there exists $\overline{z}_2$ an expansion of σ_2 such that $(A, X)[\overline{z}_1](\to_{pump} + \to_{skip})^*(A, X)[(f, Y)\overline{z}_2]$.*

This lemma is proven in Appendix G.

▶ **Proposition 8.6.** $L(\mathcal{C}_\mathcal{G}) \subseteq L^{pump,skip}(\overline{\mathcal{G}})\downarrow$.

Proof. We prove the following statement:

For all $K \in \mathbb{N}$, for all non-terminal (A, X, σ) of $\mathcal{C}_\mathcal{G}$ and $w \in T^*$, if there exists a derivation of size p from (A, X, σ) to w then for all expansion z of σ, there is a derivation *with pump and skip* from $(A, X)z$ to w' in $\overline{\mathcal{G}}$ with $w \preceq w'$.

We show this by induction on K, and distinguish cases according to the rule used in the first step of the derivation of w in $\mathcal{C}_\mathcal{G}$.

- If the rule is of the form $(A, X, \sigma) \to w$ then we can apply the corresponding rule $(A, X) \to w$ from $(A, X)z$, hence $(A, X)z \Rightarrow_{\overline{\mathcal{G}}} w$.
- If the rule is of the form $(A, X, \sigma) \to (B, X, \sigma)(C, X, \sigma)$ then there exist w_b, w_C such that $w = w_B w_c$ and $(B, X, \sigma) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w_B$ and $(C, X, \sigma) \overset{*}{\Rightarrow}_{\mathcal{C}_\mathcal{G}} w_C$. We can apply the corresponding rule $(A, X) \to (B, X)(C, X)$ from $(A, X)z$ to obtain $(B, X)[z](C, X)[z]$. By induction hypothesis, since z is an expansion of σ, we have $(B, X)[z] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'_B$ and $(C, X)[z] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'_C$ with $w_B \preceq w'_B$ and $w_C \preceq w'_C$. As a result, $(A, X, \sigma) \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'_B w'_C$, which yields the result since $w = w_B w_C \preceq w'_B w'_C$.

- If the rule is of the form $(A, X, \sigma) \to (B, Y, \texttt{push}((f, X) \blacktriangleright \sigma))$, then by Lemma 8.4 $(f, X)z$ is an expansion of $\texttt{push}((f, X) \blacktriangleright \sigma)$.
  By induction hypothesis, there exists $w' \in T^*$ such that $(B, Y)[(f, X)z] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'$ and $w \preceq w'$. Since $(A, X)[z] \Rightarrow_{\mathcal{G}} (B, Y)[(f, X)z] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'$, the induction step is proven.
- If the rule is of the form $(A, X, \sigma) \to (B, Y, \sigma')$ with $\sigma = \texttt{push}((f, Y) \blacktriangleright \sigma')$, then by Lemma 8.5 there exists z' an expansion of σ' such that $(A, X)[z] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} (A, X)[(f, Y)z']$. By induction hypothesis, there exists $w' \in T^*$ such that $(B, Y)[z'] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'$. As a consequence, $(A, X)[z] \Rightarrow_{\mathcal{G}}^{pump,skip} (A, X)[(f, Y)z'] \overset{*}{\Rightarrow}_{\mathcal{G}}^{pump,skip} w'$. The induction step is proven.

We have proven the induction. To obtain the lemma, let $w \in \mathsf{L}(\mathcal{C}_{\mathcal{G}})$. There is a derivation in $\mathcal{C}_{\mathcal{G}}$ from $(S, \mathbf{U}, \varepsilon)$ to w. Since ε is a expansion of ε, there is a derivation from $(S, \mathbf{U})$ to some w' with $w \preceq w'$. As a result, $w \in \mathsf{L}(\overline{\mathcal{G}})\downarrow$. ◀

8.3 Main result

We can conclude our proof of the other inclusion.

▶ **Proposition 8.7.** $L(\mathcal{C}_{\mathcal{G}}) \subseteq \mathsf{L}(\overline{\mathcal{G}})\downarrow$.

Proof. This is a direct corollary of Propositions 8.6 and 6.2. ◀

By Lemma 7.4, the size of a summary is bounded by an exponential in the size of $\mathcal{G}$. As a consequence, the size of $\mathcal{C}_{\mathcal{G}}$ is at most doubly exponential in the size of $\mathcal{G}$.

▶ **Theorem 8.8.** *For every indexed grammar $\mathcal{G}$ there exists a context-free grammar $\mathcal{C}_{\mathcal{G}}$ such that $\mathsf{L}(\mathcal{C}_{\mathcal{G}})\downarrow = \mathsf{L}(\mathcal{G})\downarrow$ and $\mathcal{C}_{\mathcal{G}}$ has double-exponential size in the size of $\mathcal{G}$.*

We can then compose this bound with the known bound on downward-closure of context-free languages to obtain our result.

▶ **Theorem 8.9** ([5])**.** *For all CFG $\mathcal{C}$ there is an NFA $\mathcal{A}_{\mathcal{C}}$ such that $L(\mathcal{A}_{\mathcal{C}}) = \mathsf{L}(\mathcal{C})\downarrow$ and $\mathcal{A}_{\mathcal{C}}$ has exponentially many states in the size of $\mathcal{C}$.*

Combining the two theorems above, we obtain our upper bound.

▶ **Theorem 8.10.** *For every indexed grammar $\mathcal{G}$ there exists a non-deterministic finite automaton $\mathcal{A}_{\mathcal{G}}$ such that $L(\mathcal{A}_{\mathcal{G}}) = \mathsf{L}(\mathcal{G})\downarrow$ and $\mathcal{A}_{\mathcal{G}}$ has triple-exponentially many states in the size of $\mathcal{G}$.*

9 Lower bound

In this section, we prove the lower bounds in our main results. The key ingredient is the following:

▶ **Proposition 9.1.** *Given k, we can compute in polynomial time an indexed grammar—of size polynomial in k—for the language $\{a^{\exp_3(k)}\}$.*

This implies Theorem 3.2: An NFA for $\{a^{\exp_3(k)}\}\downarrow$ clearly requires at least $\exp_3(k)$ states.

Let $k \in \mathbb{N}$.

Overview The overall strategy behind Proposition 9.1 is to construct a grammar whose (unique) derivation tree contains a full binary tree of doubly exponential depth: clearly, such a tree must have triply exponentially many leaves. Here, the challenge is to ensure that the paths have doubly exponential length.

Note that by using the stack in each non-terminal, a standard construction allows us to enforce *singly* exponentially long paths: One uses a stack with a binary alphabet of height k, and then counts up from $\mathtt{0}^k$ to $\mathtt{1}^k$, resulting in exponentially many steps. This works because when incrementing a binary expansion of the form $\mathtt{1}^\ell\mathtt{0}w$ (the least significant digit being on the left), we need to replace the prefix $\mathtt{1}^\ell\mathtt{0}$ with $\mathtt{0}^\ell\mathtt{1}$. Here, one stores the number $\ell \leq k$ in order to restore the stack height k when pushing $\mathtt{0}^\ell\mathtt{1}$.

Doing the same for stack height 2^k is not so easy: To restore a stack height of 2^k, we would need to remember (in the non-terminal) a number $\ell \leq 2^k$. In fact, enforcing a single run of length 2^{2^k} in a pushdown automaton of polynomial size is not possible.

Instead, we exploit the fact that an indexed grammar can simulate a pushdown automaton *with alternation*: We implement binary counting on a stack of height 2^k; to make sure that when replacing a prefix $\mathtt{1}^\ell\mathtt{0}$ with $\mathtt{0}^\ell\mathtt{1}$, we essentially non-deterministically push $\mathtt{0}$'s, but then use alternation to ensure that the produced stack has height exactly 2^k.

Step I: Checking the stack via alternation To this end, we introduce syntactic sugar in indexed grammars. We will use rules of the form

$$A \xrightarrow{\mathcal{A}_1,\dots,\mathcal{A}_r} B\,, \tag{1}$$

where $\mathcal{A}_1, \dots, \mathcal{A}_m$ is a list of DFAs over the stack alphabet I. Such a rule has the same effect as a rule $A \to B$, but it can only be applied to an occurrence $A[z]$ if for each $i = 1, \dots, m$, the word z has a prefix in $\mathsf{L}(\mathcal{A}_i)$. Such rules can be implemented with only polynomial overhead: Introduce non-terminals C_i, D_i for each $i = 1, \dots, r$ and also E_q for each state q in the DFAs $\mathcal{A}_1, \dots, \mathcal{A}_r$ (we assume the state sets are disjoint). Then we can simulate (1) using $A \to D_1C_1$, $D_i \to D_{i+1}C_{i+1}$ for $i = 1, \dots, r-1$, and $D_r \to B$, which split the node $A[z]$ into nodes $B[z]$ and $C_1[z], \dots, C_r[z]$. We then run $\mathcal{A}_i$ using rules $C_i \to E_q$, where q_0 is the initial state of $\mathcal{A}_i$, for each i. The non-terminals E_q simulate the DFAs: for each transition (p, a, q), we have $E_p a \to E_q$. To check acceptance, we have $E_q \to \varepsilon$ for each final state q.

Step II: Implementing a binary counter in DFAs We want to use rules (1) to check that the current stack height is 2^k, for which we construct automata $(\mathcal{A}_i)_{1\leq i\leq k}$ over a common alphabet Σ_k with the following properties:

1. Each $\mathcal{A}_i$ has two states $\mathbf{0}_i$ and $\mathbf{1}_i$, with $\mathbf{0}_i$ the initial state and $\mathbf{1}_i$ the only final one.
2. $|\Sigma_k| = k$
3. The intersection of their languages contains a single word of length 2^k

The construction is simpler if we do this for $2^k - 1$ instead of 2^k, but this is enough: We can introduce a fresh letter # and build automata $\mathcal{A}'_i$ with $\mathsf{L}(\mathcal{A}'_i) = \mathsf{L}(\mathcal{A}_i)\#$.

The idea is simply to use $\Sigma_k = \{\mathtt{inc}_1, \dots, \mathtt{inc}_k\}$. Each letter is an increment operation over a k-bit binary counter: $\mathtt{inc}_i$ should be read as "flip the ith bit from 0 to 1 and all lower bits from 1 to 0". Each automaton $\mathcal{A}_i$ keeps track of the value of the ith bit throughout that sequence of instructions.

Formally, $\mathcal{A}_i = (\{\mathbf{0}_i, \mathbf{1}_i\}, \Sigma_k, \delta_i, \mathbf{0}_i, \{\mathbf{1}_i\})$ with $\delta_i(\mathbf{0}_i, \mathtt{inc}_j) = \mathbf{1}_i$ if $j = i$, $\mathbf{0}_i$ if $j < i$, and is undefined if $j > i$. Meanwhile, $\delta_i(\mathbf{1}_i, \mathtt{inc}_j) = \mathbf{0}_i$ if $j > i$, and $\mathbf{1}_i$ if $j \leq i$. It is easy to check that there is a unique word accepted by those automata, corresponding to the only correct

sequence of instructions to increment a k-bit binary counter from 0 to $2^k - 1$, which clearly enforces a single string of length $2^k - 1$, as desired.

Step III: Indexed grammars with triply-exponential downward-closure NFA Suppose $\mathcal{A}_1, \dots, \mathcal{A}_k$ are the DFAs over Σ_k built above. Since they will check for exponential stack height, but we also need to store the binary digits on the stack, we modify them slightly. The automaton $\mathcal{B}_i$ will work over the alphabet $\{\bot\} \cup (\Sigma_k \times \{\mathtt{0}, \mathtt{1}\})$ and accept exactly the words of the form $(\alpha_1, b_1) \cdots (\alpha_m, b_m)\bot$ where $\alpha_1 \cdots \alpha_m \in \mathsf{L}(\mathcal{A}_i)$.

Note that in order for them to check for exponential stack height Consider the following grammar: $\mathcal{G}_k = (N_k, T, I_k, P_k, S)$ with

- $N_k = \{S, A, B, D, F, Z\}$
- $T = \{\mathtt{a}\}$
- $I_k = \{\bot\} \cup (\Sigma_k \times \{\mathtt{0}, \mathtt{1}\})$
- Δ_k contains the following rules:
  - $S \to Z\bot$
  - $Z \to Z(\alpha, \mathtt{0})$ for all $\alpha \in \Sigma_k$
  - $Z \xrightarrow{\mathcal{B}_1, \dots, \mathcal{B}_k} D$
  - $D \to AA$
  - $A(\alpha, \mathtt{1}) \to A$ for all $\alpha \in \Sigma_k$
  - $A(\alpha, \mathtt{0}) \to B$ for all $\alpha \in \Sigma_k$
  - $B \to Z(\alpha, \mathtt{1})$ for all $\alpha \in \Sigma_k$
  - $A\bot \to F$ and $F \to \mathtt{a}$

The grammar works as follows. Initially, it places $\bot$ on the stack and switches to Z. A non-terminal Z will then fill the stack with $\mathtt{0}$'s, which are annotated by $\alpha \in \Sigma_k$. After pushing these, it non-deterministically stops and then verifies that the stack height is actually 2^k, by checking that the word consisting of the α's is accepted by all the $\mathcal{A}_i$'s, using the rule $Z \xrightarrow{\mathcal{B}_1, \dots, \mathcal{B}_k} D$. It then splits the Z into two A's, where an increment is performed on the number encoded on the stack: It removes the prefix of the form $\mathtt{1}^\ell\mathtt{0}$ and switches to B. After this, it has to put back $\mathtt{0}^\ell\mathtt{1}$: To this end, it pushes a single $\mathtt{1}$ and switches to Z, which in turn pushes $\mathtt{0}$'s. All this repeats until in each branch, all stack contents encode the number 2^{2^k}. When this is achieved, a node $A[\mathtt{1}^{2^k}\bot]$ can be rewritten to F, and then $\mathtt{a}$. Since the nodes are split before each increment (using $D \to AA$), the final number of $\mathtt{a}$-leaves is $\exp_3(k)$, deriving $\mathtt{a}^{\exp_3(k)}$. Moreover, it is straightforward to check that $\mathtt{a}^{\exp_3(k)}$ is the only derivable word. A detailed proof can be found in Appendix I.

Computational hardness We now use methods from [17] to derive Theorem 3.4 and co-3-NEXP-hardness in Theorem 3.5 from our construction above. For this, we rely on the notion of $\Delta(f)$ language classes [17], which requires some terminology. A *transducer* is a tuple $\mathcal{T} = (Q, \Sigma, \Gamma, E, q_0, F)$, where Q is a finite set of *states*, Σ is its *input alphabet*, Γ is its *output alphabet*, $E \subseteq Q \times \Sigma^* \times \Gamma^* \times Q$ is its finite set of *edges*, $q_0 \in Q$ is its *initial state*, and $F \subseteq Q$ is its set of *final states*. It describes a relation $\mathsf{R}(\mathcal{T}) \subseteq \Sigma^* \times \Gamma^*$, namely the set of all pairs (u, v) for which there are decompositions $u = u_1 \cdots u_n$ and $v = v_1 \cdots v_n$, states $q_0, q_1, \dots, q_n$, and edges $(q_{i-1}, u_i, v_i, q_i) \in E$ for $i = 1, \dots, n$ with $q_n \in F$. For a language $L \subseteq \Sigma^*$, we write $\mathcal{T}(L) = \{v \in \Gamma^* \mid \exists u \in L \colon (u, v) \in \mathsf{R}(\mathcal{T})\}$.

A *language class* is a class of formal languages, together with some means to represent them, such as grammars or automata. A language class $\mathcal{C}$ is an *effective full trio* if for a given

language L from $\mathcal{C}$, we can effectively compute a description of $\mathcal{T}(L)$. Now suppose $f\colon \mathbb{N} \to \mathbb{N}$ is an amplifying function meaning there is a polynomial p such that $f(p(n)) \geq f(n)^2$. Then, $\mathcal{C}$ is said to be $\Delta(f)$ if (i) computing $\mathcal{T}(L)$ can be done in polynomial time and (ii) given n, one can compute a description of the language $\{a^{f(n)}\}$ in polynomial time.

In these terms, Proposition 9.1 tells us that the class of indexed languages (represented by indexed grammars) are $\Delta(\exp_3)$: Applying rational transductions to indexed languages is well-known to be possible in polynomial time.

▶ **Proposition 9.2.** *The indexed languages (represented by indexed grammars) are* $\Delta(\exp_3)$.

We will also need the notion of simple substitutions. For alphabets Σ, Γ, a *substitution* is a map $\sigma\colon \Sigma \to 2^{\Gamma^*}$ that replaces each letter in Σ by a language over Γ. For language $L \subseteq \Sigma^*$, the language $\sigma(L)$ is defined in the obvious way. The substitution σ is said to be *simple for* $L \subseteq \Sigma^*$ if $\Sigma \subseteq \Gamma$ and there is a letter $a \in \Sigma$ such that $\sigma(a') = \{a'\}$ for each $a' \in \Sigma \setminus \{a\}$. We say that a language class $\mathcal{C}$ is *closed under simple subsititutions* if for any given L from $\mathcal{C}$, and any simple substitution σ for L, the language $\sigma(L)$ belongs to $\mathcal{C}$, and a representation can be computed in polynomial time. It is easy to see that the indexed languages are closed under simple substitutions. In [17, Theorem 15], it is shown that downward closure inclusion and equivalence are both coTIME(t)-hard for any language class that is $\Delta(t)$ and closed under simple substitutions. Hence, Proposition 9.1 implies co-3-NEXP-hardness of downward closure inclusion and equivalence.

DFA size We can also use the notion of $\Delta(f)$ language classes to obtain a lower bound on the size of DFAs for downward closures of indexed languages. To this end, we make a slightly more general observation:

▶ **Proposition 9.3.** *Let* $f\colon \mathbb{N} \to \mathbb{N}$ *be a function such that a language class* $\mathcal{C}$ *is* $\Delta(f)$ *and let* $\mathcal{C}$ *be closed under simple substitutions. Then there is a family languages* $(L_k)_{k\geq 1}$ *with polynomial description sizes such that any DFA for* $L_k{\downarrow}$ *requires at least* $2^{f(k)}$ *states.*

Proof. We claim that we can construct a representation of

$$L_k = \{uv \mid u, v \in \{\mathtt{0},\mathtt{1}\}^*,\ |u| = |v| = f(k),\ u \neq v\}$$

in polynomial time. Note that a DFA for $L_k{\downarrow}$ requires at least $2^{f(k)}$ states: After reading distinct prefixes $u, u' \in \{\mathtt{0},\mathtt{1}\}^*$ of length $f(k)$, the DFA must enter distinct states, as otherwise, it would accept uu, which does not belong to $L_k{\downarrow}$.

To construct L_k for given $k \in \mathbb{N}$, we begin by building a representation of $\{\mathtt{a}^{f(k)}\}$. Using a transducer, we then insert a single occurrence of a letter $\mathtt{b}$ into every word, and then substitute this b with $\{\mathtt{ba}^{f(k)}\mathtt{c}\}$. This yields the language $\{\mathtt{a}^r\mathtt{ba}^{f(k)}\mathtt{ca}^s \mid r + s = f(k)\}$. Using another transducer, we can remove two occurrences of a within $\mathtt{a}^r$ and $\mathtt{a}^s$ to obtain $\{\mathtt{a}^r\mathtt{ba}^{f(k)}\mathtt{ca}^s \mid r + s = f(k) - 2\}$. A final transducer then replaces (i) each $\mathtt{a}$ with $\mathtt{0}$ or $\mathtt{1}$ and (ii) $\mathtt{b}$ and $\mathtt{c}$ by distinct letters in $\{\mathtt{0},\mathtt{1}\}$. This results in the language L_k. ◀

Now, Proposition 9.3 and Proposition 9.2 together directly imply Theorem 3.4.

10 Conclusion

We have established tight bounds on the size of an automaton recognizing the downward-closure of an indexed grammar. We rely on an algebraic abstraction of stack contents to translate indexed grammars into context-free ones while preserving the downward-closure. We suspect that this approach can be extended to the whole hierarchy of higher-order pushdown automata.

References

1 Parosh Aziz Abdulla, Luc Boasson, and Ahmed Bouajjani. Effective lossy queue languages. In Fernando Orejas, Paul G. Spirakis, and Jan van Leeuwen, editors, *Automata, Languages and Programming, 28th International Colloquium, ICALP 2001, Crete, Greece, July 8-12, 2001, Proceedings*, volume 2076 of *Lecture Notes in Computer Science*, pages 639–651. Springer, 2001. doi:10.1007/3-540-48224-5_53.

2 Alfred V. Aho. Indexed grammars - an extension of context-free grammars. *J. ACM*, 15(4):647–671, 1968. doi:10.1145/321479.321488.

3 Mohamed Faouzi Atig, Ahmed Bouajjani, and Shaz Qadeer. Context-bounded analysis for concurrent programs with dynamic creation of threads. *Log. Methods Comput. Sci.*, 7(4), 2011. doi:10.2168/LMCS-7(4:4)2011.

4 Noam Chomsky. On certain formal properties of grammars. *Inf. Control.*, 2(2):137–167, 1959. doi:10.1016/S0019-9958(59)90362-6.

5 Bruno Courcelle. On constructing obstruction sets of words. *Bull. EATCS*, 44:178–186, 1991.

6 Wojciech Czerwinski, Wim Martens, Lorijn van Rooijen, and Marc Zeitoun. A note on decidable separability by piecewise testable languages. In Adrian Kosowski and Igor Walukiewicz, editors, *Fundamentals of Computation Theory - 20th International Symposium, FCT 2015, Gdańsk, Poland, August 17-19, 2015, Proceedings*, volume 9210 of *Lecture Notes in Computer Science*, pages 173–185. Springer, 2015. doi:10.1007/978-3-319-22177-9_14.

7 Michael J. Fischer. Grammars with macro-like productions. In *9th Annual Symposium on Switching and Automata Theory, Schenectady, New York, USA, October 15-18, 1968*, pages 131–142. IEEE Computer Society, 1968. doi:10.1109/SWAT.1968.12.

8 Hugo Gimbert, Corto Mascle, and Patrick Totzke. Optimal sequential flows. *arXiv preprint arXiv:2511.13806*, 2025.

9 Graham Higman. Ordering by divisibility in abstract algebras. *Proceedings of the London Mathematical Society*, s3-2(1):326–336, 1952. doi:https://doi.org/10.1112/plms/s3-2.1.326.

10 Ismaël Jecker. A Ramsey Theorem for Finite Monoids. In Markus Bläser and Benjamin Monmege, editors, *38th International Symposium on Theoretical Aspects of Computer Science, STACS 2021, March 16-19, 2021, Saarbrücken, Germany (Virtual Conference)*, volume 187 of *LIPIcs*, pages 44:1–44:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. URL: https://doi.org/10.4230/LIPIcs.STACS.2021.44, doi:10.4230/LIPICS.STACS.2021.44.

11 Salvatore La Torre, Anca Muscholl, and Igor Walukiewicz. Safety of parametrized asynchronous shared-memory systems is almost always decidable. In Luca Aceto and David de Frutos-Escrig, editors, *26th International Conference on Concurrency Theory, CONCUR 2015, Madrid, Spain, September 1-4, 2015*, volume 42 of *LIPIcs*, pages 72–84. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015. URL: https://doi.org/10.4230/LIPIcs.CONCUR.2015.72, doi:10.4230/LIPICS.CONCUR.2015.72.

12 Aleksandr Nikolaevich Maslov. Multilevel stack automata. *Problemy peredachi informatsii*, 12(1):55–62, 1976.

13 Robert McNaughton. Varieties of formal languages (J. e. pin; a. howie, trans.). *SIAM Rev.*, 31(2):347–348, 1989. doi:10.1137/1031081.

14 Jacques Sakarovitch. *Elements of Automata Theory*. Cambridge University Press, 2009. URL: http://www.cambridge.org/uk/catalogue/catalogue.asp?isbn=9780521844253.

15 Imre Simon. Factorization forests of finite height. *Theor. Comput. Sci.*, 72(1):65–94, 1990. doi:10.1016/0304-3975(90)90047-L.

16 Georg Zetzsche. An approach to computing downward closures. In Magnús M. Halldórsson, Kazuo Iwama, Naoki Kobayashi, and Bettina Speckmann, editors, *Automata, Languages, and Programming - 42nd International Colloquium, ICALP 2015, Kyoto, Japan, July 6-10, 2015, Proceedings, Part II*, volume 9135 of *Lecture Notes in Computer Science*, pages 440–451. Springer, 2015. doi:10.1007/978-3-662-47666-6_35.

788 **17** Georg Zetzsche. The complexity of downward closure comparisons. In Ioannis Chatzigiannakis,
789 Michael Mitzenmacher, Yuval Rabani, and Davide Sangiorgi, editors, *43rd International*
790 *Colloquium on Automata, Languages, and Programming, ICALP 2016, Rome, Italy, July*
791 *11-15, 2016*, volume 55 of *LIPIcs*, pages 123:1–123:14. Schloss Dagstuhl - Leibniz-Zentrum für
792 Informatik, 2016. doi:10.4230/LIPICS.ICALP.2016.123.

A Normalising indexed grammars

When describing indexed grammars we sometimes use production rules of the form not allowed by our definition of indexed grammar

(i) $Af \to u$ with $u \in (N \cup T)^* \setminus N$
(ii) $A \to u$ with $u \in (N \cup T)^* \setminus (N^2 \cup T^*)$

with u any element of $(N \cup T)^*$.

We now formally define how these rules should be eliminated to obtain an indexed grammar as in Definition 2.2.

We start by eliminating rules of the first type: we replace each rule $Af \to u$ with $u \in (N \cup T)^* \setminus N$ by two rules $Af \to A'$ and $A' \to u$, with A' a fresh non-terminal.

It remains to eliminate rules of the form $A \to u$ with $u \in (N \cup T)^* \setminus (N^2 \cup T^*)$. If $u = B \in N$ then replace the rule with $A \to BC$ and $C \to \varepsilon$ with C a fresh non-terminal. Otherwise, decompose u as $u = w_0 A_1 w_1 \dots A_k w_k$ with $A_1, \dots, A_k \in N$ and $w_0, \dots, w_k \in T^*$. Introduce fresh non-terminals $B_1, \cdots, B_k, C_1, \dots, C_{k-1}$. We replace $A \to u$ with rules

- $A \to W_0 B_1$,
- $W_i \to w_i$ for all $i \in \{0, \dots, k\}$,
- $B_i \to A_i C_i$ for all $i \in \{1, \dots, k-1\}$,
- $C_i \to W_i B_{i+1}$ for all $i \in \{1, \dots, k-1\}$,
- $B_k \to A_k W_k$

Note that $\sum_{i=0}^{k} |w_i| \leq |u|$ and $k \leq |u|$. Hence, each such rule $A \to u$ is replaced by a set of at most $3|u|+1$ rules, introducing at most $2|u|-1$ non-terminals whose lengths sum up to at most $8|u|+6$. As a consequence, each rule of the form $Af \to u$ is replaced by a set of at most $3|u|+2$ rules whose lengths sum up to at most $8|u|+8$.

B Proof of Lemma 5.1

▶ **Lemma 5.1.** *Let $\overline{z} = (f_n, X_n) \cdots (f_1, X_1) \in \overline{I}^*$ be a non-empty stack. The following are equivalent:*

1. *$\overline{z}$ is feasible*
2. *$\varphi(\overline{z}) \neq \bot$*
3. *for all $i > 1$, $X_i = f_i \cdot X_{i-1}$, $\beta(f_{i-1}) \ \mathcal{R}_{X_i} \ \alpha(f_i)$.*

Proof. From the definitions of $\mathbb{S}$ and φ, one sees that property 3 is necessary and sufficient to ensure that no product of two consecutive factors in $\varphi(\overline{z}) = \prod_{i=1}^{n} \varphi(f_i, X_i)$ is equal to $\bot$. It is also clear from the definitions that $\prod_{i=1}^{n} \varphi(f_i, X_i) = \bot$ if and only if two consecutive factors multiply to equal $\bot$, so we immediately obtain $2 \Leftrightarrow 3$.

We now show that $1 \Rightarrow 3$. Let $\overline{z}$ be feasible, so by definition we have a derivation

$$(\alpha(f_1), X_1) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(\beta(f_n), f_n \cdot X_n)[\overline{z}]v. \tag{$*$}$$

We proceed by induction on the derivation length. If $(*)$ has length one, then it must be of the form $(\alpha(f_1), X_1) \Rightarrow_{\overline{\mathcal{G}}} (\beta(f_1), f_1 \cdot X_1)[(f_1, X_1)]$, whence 3 holds trivially. We now assume that the length is greater than one, and that the first operation is of the form $(\alpha(f_1), X_1) \to (B, X_1)(C, X_1)$. In this case, we may assume without loss of generality that (B, X_1) derives $u'(\beta(f_n), f_n \cdot X_n)[\overline{z}]v'$ for some $u', v' \in \mathbf{SF}$. Since (f_1, X_1) must eventually be pushed (and $(\alpha(f_1), X_1)$ is the only nonterminal which allows this), it follows that

$B\ \mathcal{R}_{X_1}\ \alpha(f_1)$, and that there is a derivation $(\alpha(f_1), X_1) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u''(\beta(f_n), f_n \cdot X_n)[\overline{z}]v''$ for some $u'', v'' \in \mathbf{SF}$ that is strictly shorter than $(*)$. Hence, 3 holds by induction.

Now let us assume that the length of $(*)$ is greater than one, and that the first operation is a push (the only remaining possibility). The push operation must be of the form

$$(\alpha(f_1), X_1) \Rightarrow_{\overline{\mathcal{G}}} (\beta(f_1), f_1 \cdot X_1)[(f_1, X_1)].$$

If $f_1 \cdot X_1 \neq X_2$, then it is easy to see that the right hand side cannot derive any term which pushes (f_2, X_2). Hence, we have

$$X_2 = f_1 \cdot X_1 \text{ and } \beta(f_1)\ \mathcal{R}_{X_2}\ \alpha(f_2), \tag{$**$}$$

and there is a derivation $(\alpha(f_2), X_2)[(f_1, X_1)] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u'(\beta(f_n), f_n \cdot X_n)[\overline{z}]v'$. Letting $\overline{z}' = (f_n, X_n) \cdots (f_2, X_2)$, this implies that $\overline{z}'$ is feasible with a derivation

$$(\alpha(f_2), X_2) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u'(\beta(f_n), f_n \cdot X_n)[\overline{z}']v'$$

that is strictly shorter than $(*)$. Hence, $\overline{z}'$ satisfies 3 by induction, and with $(**)$ we immediately obtain 3 for $\overline{z}$.

Finally, we show that $3 \Rightarrow 1$. Let us assume that $\overline{z}$ satisfies 3. Then for $i = 1, \ldots, n-1$ we have

$$(\alpha(f_i), X_i) \Rightarrow_{\overline{\mathcal{G}}} (\beta(f_i), X_{i+1})[(f_i, X_i)] \text{ and } (\beta(f_i), X_{i+1}) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u_i(\alpha(f_{i+1}), X_{i+1})v_i$$

for some $u_i, v_i \in \mathbf{SF}$, as well as $(\alpha(f_n), X_n) \Rightarrow_{\overline{\mathcal{G}}} (\beta(f_n), f_n \cdot X_n)[(f_n, X_n)]$. It is clear that we may combine these derivations to obtain $(\alpha(f_1), X_1) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(\beta(f_n), f_n \cdot X_n)[\overline{z}]v$, proving that $\overline{z}$ is feasible. ◀

C Proof of Lemma 5.2

▶ **Lemma 5.2.** *Let $z \in I^*$, $X \subseteq N$ with $|z| \geq 1$ and let $(B, Y, M, A, X) = \varphi(\overline{z}^X)$, we have, for all $C \in X$ and $D \in Y$,*

$$C\ \mathcal{R}_X\ A \text{ and } B\ \mathcal{R}_Y\ D$$
$$\Updownarrow$$
$$\text{there exist } u, v \in \mathbf{SF} \text{ such that } (C, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(D, Y)[\overline{z}^X]v.$$

Proof. From the first condition, we have derivations

$$(C, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u_C(A, X)v_C \text{ and } (B, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u_D(D, X)v_D$$

for some $u_C, v_C, u_D, u_D \in \mathbf{SF}$. Let $z = f_n \cdots f_1$. From the definition of φ (and the fact that $\varphi(\overline{z}^X) \neq \bot$) it follows easily that $A = \alpha(f_1)$, $B = \beta(f_n)$ and $Y = z \cdot X$. Hence, Lemma 5.1 ensures that there is a derivation

$$(A, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u'(B, Y)[\overline{z}^X]v'$$

for some $u', v' \in \mathbf{SF}$, which we combine with the derivations above to obtain

$$(C, X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(D, Y)[\overline{z}^X]v,$$

proving one direction.

For the other implication, suppose such a derivation exists. Eventually, (f_1, X) must be pushed onto an empty stack, and since (A, X) is the only non-terminal which facilitates this operation, it follows that $C\ \mathcal{R}_X\ A$. Similarly, the derivation must eventually push the topmost symbol in $\overline{z}^X$, and the only non-terminal which can result from this operation is (B, Y). This implies that $(B, Y)[\overline{z}^X] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(D, Y)[\overline{z}^X]v$, hence $B\ \mathcal{R}_Y\ D$, completing the proof. ◀

D Proof of Lemma 5.3

▶ **Lemma 5.3.** *Let $z \in I^*$, $X \subseteq N$ with $|z| \geq 1$ and let $(B, Y, M, A, X) = \varphi(\overline{z}^X)$, we have*

$$M(D, C) = \top$$
$$\Updownarrow$$
$$C \in X,\ D \in Y \text{ and there exist } u, v \in (X \cup T)^* \text{ such that } D[z] \overset{*}{\Rightarrow}_{\mathcal{G}} uCv.$$

Proof. We proceed by induction on z. If $|z| = 1$ then we have $\overline{z}^X = (f, X)$ with $\varphi(f, X) = (B, Y, M, A, X)$. Then the equivalence holds simply by definition.

If $|z| > 1$ then let $z = fz'$. Let $Z = z' \cdot X$, we have $\overline{z}^X = (f, Z)\overline{z'}^X$. Define $(B_f, Y, M_f, A_f, Z) = \varphi(f, Z)$ and $(B', Z, M', A', X) = \varphi(\overline{z'}^X)$. Note that $M = M_f M'$ We prove the two directions separately.

$\Rightarrow$ Suppose $M(D, C) = \top$. Then there exists E such that $\mu_f(D, E) = \mu'(E, C) = \top$.
By induction hypothesis applied to f we have: $D \in Y$, $E \in Z$ and there exist $u_1, v_1 \in (Z \cup T)^*$ such that $D[f] \overset{*}{\Rightarrow}_{\mathcal{G}} u_1 E v_1$.
Furthermore, by induction hypothesis applied to z' we have $C \in X$ and there exist $u_2, v_2 \in (X \cup T)^*$ such that $E[z'] \overset{*}{\Rightarrow}_{\mathcal{G}} u_2 C v_2$.
Since $Z = z' \cdot X$, there exist $u_3, v_3 \in (X \cup T)^*$ such that $u_1^{z'} \overset{*}{\Rightarrow}_{\mathcal{G}} u_3$ and $v_1^{z'} \overset{*}{\Rightarrow}_{\mathcal{G}} v_3$.
Here $u_1^{z'}$ and $v_1^{z'}$ denote the sentential forms u_1 and v_1 where z' has been pushed on top of the stack of every non-terminal.
Combining those facts, we get $D[z] \overset{*}{\Rightarrow}_{\mathcal{G}} u_1^{z'} E[z'] v_1^{z'} \overset{*}{\Rightarrow}_{\mathcal{G}} u_3 E[z'] v_3 \overset{*}{\Rightarrow}_{\mathcal{G}} u_3 u_2 C v_2 v_3$.

$\Leftarrow$ Suppose $D \in Y$, $C \in X$ and there exist $u, v \in (X \cup T)^*$ such that $D[z] \overset{*}{\Rightarrow}_{\mathcal{G}} uCv$.
Then we have $D[f] \overset{*}{\Rightarrow}_{\mathcal{G}} u_1 E v_1$, $E[z'] \overset{*}{\Rightarrow}_{\mathcal{G}} u_2 C v_2$, $u_1^{z'} \overset{*}{\Rightarrow} \mathcal{G} u_3$ and $v_1^{z'} \overset{*}{\Rightarrow} \mathcal{G} v_3$ for some $u_1, u_2, u_3, v_1, v_2, v_3$ such that $u = u_3 u_2$ and $v = v_2 v_3$.
As a consequence, we have $u_2, u_3, v_2, v_3 \in (X \cup T)^*$. Since $z' \cdot X = Z$, we get that $u_1, v_1 \in (Z \cup T)^*$. Since $C \in X$, the derivation $E[z'] \overset{*}{\Rightarrow}_{\mathcal{G}} u_2 C v_2 \in (X \cup T)^*$ proves that $E \in z' \cdot X = Z$.
By induction hypothesis, we have $M'(D, E) = M_f(E, C) = \top$ and thus $M(D, C) = \top$.

◀

E Proof of Lemma 7.4

In this section we prove the following statement.

▶ **Lemma 7.4.** *For all $z \in \overline{I}^*$, $\mathtt{push}(z \blacktriangleright \varepsilon)$ is of size at most exponential in N.*

Its proof is very similar to the one of Theorem 26 in [8].
Let us start by recalling some facts on Green relations.

▶ **Lemma E.1** ([13])**.** *In a finite semigroup $\mathbf{S}$, every $\mathcal{H}$-class contains at most one idempotent.*

▶ **Lemma E.2** ([13])**.** *In a finite semigroup $\mathbf{S}$, for all $x, y \in \mathbf{S}$, if $x \mathcal{J} y$ and $x \leq_{\mathcal{L}} y$ (resp. $x \leq_{\mathcal{R}} y$) then $x \mathcal{L} y$ (resp. $x \mathcal{R} y$).*

Define the *Ramsey function* of $\mathbf{S}$ as follows: for all $k \in \mathbb{N}$, $\mathcal{R}_{\mathbf{S}}(k)$ is the minimal n such that every word of length n contains a factor u such that $\psi_{\mathbf{S}}(u)$ is an idempotent.

894 ▶ **Theorem E.3** ([10], Theorem 1). *For all finite semigroup*[2] $\mathbf{S}$*, for all* $k \in \mathbb{N}$*,*

895 $\mathcal{R}_{\mathbf{S}}(k) \leq (k|\mathbf{S}|^4)^{JL(\mathbf{S})}$.

896 ▶ **Theorem E.4** ([10], Theorem 2). *The regular* $\mathcal{J}$*-length of* $\mathbb{M}$ *is* $JL(\mathbb{M}) = \frac{N^2+N+2}{2}$.

897 ▶ **Theorem E.5.** *The regular* $\mathcal{J}$*-length of* $\mathbb{S}$ *is at most* $\frac{(N^2+N+2)}{2} + 1$.

898 **Proof.** Let us start by using another definition of the regular $\mathcal{J}$-length. By [10], the regular
899 $\mathcal{J}$-length of $\mathbb{S}$ is the largest m such that there is an injective homomorphism from the max
900 monoid $(\{1, \ldots, m\}, \max, m)$ to $\mathbb{S}$.

901 Let $m \in \mathbb{N}$, let $\theta : \{1, \ldots, m\} \to \mathbb{S}$ be such a homomorphism. We must show that
902 $m \leq \frac{(N^2+N+2)}{2} + 1$.

903 If $\bot$ is in the image of θ then $\theta(m) = \bot$. Since all i in $\{1, \ldots, m-1\}$ are idempotents,
904 the image of each i by θ must also be an idempotent. As a result, we can set $\theta(i) =$
905 $(B_i, X_i, M_i, A_i, X_i)$ for all $i < m$, with $M_i^2 = M_i$.

906 Furthermore, for all $i < j < m$, since $\max(i, j) = j$, we must have $(B_j, Y_j, M_j, A_j, X_j) \cdot$
907 $(B_i, Y_i, M_i, A_i, X_i) = (B_i, Y_i, M_i, A_i, X_i) \cdot (B_j, Y_j, M_j, A_j, X_j) = (B_j, Y_j, M_j, A_j, X_j)$. We
908 infer that $X_j = X_i$, $B_j = B_i$ and $A_j = A_i$ for all $i < j$. Since θ is injective, $M_1, \ldots, M_{m-1}$
909 must be distinct.

910 Define the function $\theta' : \{1, \ldots, m-1\}$ mapping each i to M'_i. It suffices to observe
911 that θ_p is an injective homomorphism from the max monoid of size $m-1$ to $\mathbb{M}$. As a
912 consequence, we have $m - 1 \leq JL(\mathbb{M})$. By Theorem E.4, we have $JL(\mathbb{M}) \leq \frac{N^2+N+2}{2}$. As a
913 result, $m \leq \frac{(N^2+N+2)}{2} + 1$. ◀

914 Observe that the size of $\mathbb{S}$ is bounded by $N^2 2^{4N^2+8N}$. Define $K = ((N+1)N^2 2^{4N^2+8N})^{\frac{(N^2+N+2)}{2}+1}$.
915 By combining the results above, we obtain the following corollary.

916 ▶ **Corollary E.6.** *Let* $z \in \mathbb{S}^*$ *with* $|z| \geq K$*, there exist* $v_0, \ldots, v_N \in \mathbb{S}^*$ *and* $e \in \mathbf{Idem}(\mathbb{S})$ *such that*

- 918 $v_0 \ldots v_N$ *is a factor of* z
- 919 $\varphi(v_0) = \cdots = \varphi(v_N) = e$

920 In what follows we distinguish the size of an h-summary/h-block from its *length*, which is
921 simply its length as a word of h-atoms, plus one (for the $(h-1)$-summary at the start).

922 ▶ **Lemma E.7.** *For all* $z \in \overline{I}^*$, $\texttt{push}(z \blacktriangleright \varepsilon) = \sigma' u B_1 \ldots B_K$ *is such that* u *and all* B_i *have*
923 *length at most* K*.*

924 **Proof.** If $\texttt{push}(z \blacktriangleright \varepsilon) = \sigma' u B_1 \ldots B_k$ then u cannot have an infix of the form $v_0 \ldots v_N$ with
925 $\varphi(v_i) = e$ for all i, for any $e \in \mathbf{Idem}(\mathbb{S})$. This is because when such a pattern appears u is
926 turned into an h-block.

927 As a consequence, by Corollary E.6, u has length at most $K - 1$.

928 Therefore, since all blocks are created by concatenating an h-atom with such a u, they
929 have length at most K. ◀

930 ▶ **Lemma E.8.** *For all* $z \in \overline{I}^*$, $\texttt{push}(z \blacktriangleright \varepsilon)$ *has at most* $|\mathbb{S}|^{4JL(\mathbb{S})+1}$ *blocks.*

[2] The theorem is stated for finite monoids in [10], but it is straightforward to adapt it to finite semigroups.

Proof. Let $\sigma = \sigma' u B_1 \dots B_m$ be an h-summary. For each i let $B_i = e_i^+ v_{i,1}, \dots, v_{i,N} w_i$ and for each $i < j$ define $\alpha_i = \psi_{\mathbb{S}}(v_{i,1} \dots v_{i,N} w_i B_{i+1} \cdots B_{j-1} e_j)$.

Suppose by contradiction that $m > |\mathbb{S}|^{4JL(\mathbb{S})+1}$, then by pigeonhole principle there exist $e \in \mathbf{Idem}(\mathbb{M})$ and $i_1 < \cdots < i_p \in \{1, \dots, K\}$ such that $p > (|\mathbb{S}|)^{4JL(\mathbb{S})}$ and $e_{i_k} = e$ for all k.

Then by Theorem E.3, there exist k, ℓ such that $\psi_{\mathbb{S}}(\alpha_{i_k} \dots \alpha_{i_\ell})$ is an idempotent e'.

Since σ is an h-summary, we have $\mathbf{height}(e') = h = \mathbf{height}(\varphi_{\mathbb{M}}(\alpha_i \dots \alpha_j)) = \mathbf{height}(e)$. Since $e' \leq_{\mathcal{J}} e$, we have $e \mathcal{J} e_i$. Furthermore, since $e' \leq_{\mathcal{R}} \alpha_{i_k} \leq_{\mathcal{R}} \varphi_{\mathbb{M}}(\pi(v_{i_k,1})) = e$, we have $e' \leq_{\mathcal{R}} e$ and thus $e \mathcal{R} e'$ by Lemma E.1. Similarly, since $e' \leq_{\mathcal{L}} \alpha_{i_\ell} \leq_{\mathcal{L}} e$ we have $e' \leq_{\mathcal{L}} e$ and thus $e \mathcal{L} e'$.

We obtain $e \mathcal{H} e'$. By Lemma E.2, this implies $e = e'$. This is a contradiction since then the blocks from B_{i_k} to B_{i_ℓ} should have been merged when B_{i_k} was created. As a result, we must have $m \leq |\mathbb{S}|^{4JL(\mathbb{S})+1}$. ◀

We now have all necessary tools to show Lemma 7.4.

Proof of Lemma 7.4. We show that for all z and h, if $\mathtt{push}(z \blacktriangleright \varepsilon)$ is an h-summary then it has size at most $((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^h$.

We do an induction on h. The property trivially holds for $h = 0$. Let $h > 0$, suppose the property holds for $h - 1$.

Let $\mathtt{push}(z \blacktriangleright \varepsilon) = \sigma' u B_1 \dots B_m$ By Lemma E.7 we have $m \leq |\mathbb{S}|^{4JL(\mathbb{S})+1}$. By Lemma E.8 we have $|u| \leq K$ and $|B_i| \leq K$ for all i.

Therefore $\mathtt{push}(z \blacktriangleright \varepsilon)$ has length at most $(|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1$

Let $a\sigma''$ be an h-atom appearing in z, with σ'' an $h-1$-summary. Observe that there must be a strict infix z'' of z such that $\mathtt{push}(z'' \blacktriangleright \varepsilon) = \sigma''$. Hence by induction hypothesis σ'' has size at most $((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^{h-1}$. Thus every h-atom has size at most $((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^{h-1} + 1 \leq 2((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^{h-1}$. With the bound on its length, we conclude that $\mathtt{push}(z \blacktriangleright \varepsilon)$ has size at most

$$2((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^{h-1}((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1) \leq ((|\mathbb{S}|^{4JL(\mathbb{S})+1} + 1)K + 1)^h.$$

We obtain the result by applying this bound with $h = JL(\mathbb{S})$, which is at most $\frac{(N^2+N+2)}{2} + 1$ by Theorem E.5. ◀

F Proof of Lemma 8.4

▶ **Lemma 8.4.** *Let $(A, X, \sigma_1) \to (B, Y, \sigma_2)$ be a rule of $\mathcal{C}_{\mathcal{G}}$ with $\sigma_2 = \mathtt{push}((f, X) \blacktriangleright \sigma_1)$. Let $\overline{z}_1$ an expansion of σ_1. Then $(f, X)\overline{z}_2$ is an expansion of σ_2.*

Proof. We prove this statement by induction on the height of σ_2. If σ_2 has height 0 then it is ε, which contradicts $\sigma_2 = \mathtt{push}((f, X) \blacktriangleright \sigma_1)$.

If σ_2 has height $h > 0$ then we distinguish cases according to its shape. In most cases we will simply show that $(f, Y)\overline{z}_1$ is a expansion of σ_2. In case a. we will use the induction hypothesis, and in case A. we will actually apply a pump rule to obtain a suitable $\overline{z}_2$.

Let $\sigma_1 = \sigma' u B_1 \dots B_k$. By definition z must be of the form $z' z^u z_1 \dots z_k$ with z', z^u, $z_1, \dots, z_k$ expansions of respectively σ', u, $B_1, \dots, B_k$.

1. If $\mathbf{height}((f, X)\sigma_1) > h$ then $\sigma_2 = (f, X)\sigma_1$. Then by definition $(f, X)\overline{z}_1$ must be a expansion of σ_2.
2. Otherwise, we have $\mathbf{height}((f, X)\sigma_1) = h$

a. if $\mathbf{height}((f,X)\sigma') < h$ then $\sigma_2 = (\mathtt{push}((f,X) \blacktriangleright \sigma'))uB_1 \dots B_k$.
By induction hypothesis, there exists z'' such that $(B,Y)[z'] \to^*_{pump} (B,Y)[z'']$ and z'' is a expansion of $\mathtt{push}((f,X) \blacktriangleright \sigma')$. Therefore, we have $(B,Y)[z'z^u z_1 \dots z_k] \to^*_{pump} (B,Y)[z''z^u z_1 \dots z_k]$. Further, $z''z^u z_1 \dots z_k$ is a expansion of $\sigma_2 = \mathtt{push}((f,X) \blacktriangleright \sigma')u\sigma_1 \dots \sigma_k$.

b. Otherwise, $\mathbf{height}((f,X)\sigma') = h$ and $(f,X)\sigma'$ is an h-atom.

i. If $((f,X)\sigma')u$ is of the form $u_1 \dots u_N v_1 \dots v_N w$ with $\varphi(u_i) = \varphi(v_i) = e$ for all i, for some $e \in \mathbf{Idem}(\mathbb{S})$ then $(f,X)z' = z_1^u \dots z_N^u z_1^v \dots z_N^v z^w$, where z_j^u and z_j^v are expansions of u_j and v_j for all j, and z^w of w. We have two cases.

A. If there exists j such that B_j is of the form $u'_1 \dots u'_N e^+ v'_1 \dots v'_N w'$ and $\varphi(v_1 \dots v_N w B_1 \dots B_{j-1} u'_1 \dots u'_N) = e$ then we have $\sigma_2 = (u_1 \dots u_N e^+ v'_1 \dots v'_N w')B_{j+1} \dots B_k$. In that case, we also have $z_j = z_1^{u'} \dots z_N^{u'} z'_e z_1^{v'} \dots z_N^{v'} z^{w'}$, where $z_j^{u'}$ and $z_j^{v'}$ are expansions of u_j and v_j for all j, z'_e is a expansion of e^+ and $z^{w'}$ of w'.
Furthermore, since $\varphi(v_1 \dots v_N B_1 \dots B_{j-1} u'_1 \dots u'_N) = e$, we have $\varphi(z_1^v \dots z_N^v z^w z_1 \dots z_{j-1} z_1^{u'} \dots z_N^{u'}) = e$. Since z'_e is a expansion of e^+, so is $z_e = z_1^v \dots z_N^v z^w z_1 \dots z_{j-1} z_1^{u'} \dots z_N^{u'} z'_e$. Since $(f,X)\overline{z}_1 = z_1^u \dots z_N^u z_e z_1^{v'} \dots z_N^{v'} z^{w'} z_{j+1} \dots z_k$, $(f,X)\overline{z}_1$ is a expansion of $\sigma_2 = u_1 \dots u_N e^+ v'_1 \dots v'_N w' B_{j+1} \dots B_k$

B. Otherwise $\sigma_2 = (u_1 \dots u_N e^+ v_1 \dots v_N w)B_1 \dots B_k$. Let z'_e be a expansion of e^+. Observe that since $\varphi(u_1 \dots u_N) = e$, $\varphi(z_1^u \dots z_N^u) = e$. Further, $\varphi(z'_e) = e$, and thus we have $(B,Y)[(f,X)\overline{z}_1] = (B,Y)[z_1^u \dots z_N^u z_1^v \dots z_N^v z^w z_1 \dots z_k] \to_{pump} (B,Y)[z_1^u \dots z_N^u z'_e z_1^u \dots z_N^u z_1^v \dots z_N^v z^w z_1 \dots z_k]$. Since z'_e is a expansion of e^+, so is $z'_e z_1^u \dots z_N^u$. As a consequence, the resulting stack content $z_1^u \dots z_N^u z'_e z_1^u \dots z_N^u z_1^v \dots z_N^v z^w z_1 \dots z_k$ is a expansion of σ_2.

ii. Otherwise, we have $\sigma_2 = ((f,X)\sigma')uB_1 \dots B_k$ and thus $(f,X)\overline{z}_1$ is a expansion of σ_2.

◀

G Proof of Lemma 8.5

▶ **Lemma 8.5.** *Let $(A,X,\sigma_1) \to (B,Y,\sigma_2)$ be a rule of $\mathcal{C}_\mathcal{G}$ with $\sigma_2 \in \mathtt{pop}((f,Y) \blacktriangleleft \sigma_1)$. Let $\overline{z}_1$ an expansion of σ_1. Then there exists $\overline{z}_2$ an expansion of σ_2 such that $(A,X)[\overline{z}_1](\to_{pump} + \to_{skip})^*(A,X)[(f,Y)\overline{z}_2]$.*

Proof. We prove this statement by induction on the height of σ_1. If σ_1 has height 0 then it is ε and $\mathtt{pop}((f,Y) \blacktriangleleft \sigma_1)$ is empty.

If σ_1 has height $h > 0$ then we distinguish cases according to its shape. In most cases we will simply show that $\overline{z}_1$ is of the form $(f,Y)\overline{z}_2$ with $\overline{z}_2$ a expansion of σ_2. Let $\sigma_2 = \sigma' u B_1 \dots B_k$.

1. If $\mathbf{height}((f,Y)\sigma_2) > h$ then $\sigma_1 = (f,Y)\sigma_2$. Then by definition $\overline{z}_1$ must be of the form $(f,Y)\overline{z}_2$ with $\overline{z}_2$ an expansion of σ_2.
2. Otherwise, we have $\mathbf{height}((f,Y)\sigma_2) = h$

a. if $\mathbf{height}((f,Y)\sigma') < h$ then $\sigma_1 = (\mathtt{push}((f,Y) \blacktriangleright \sigma'))uB_1 \dots B_k$. Then we have $\overline{z}_1 = z''z^u z_1 \dots z_k$ with z'' a expansion of $\mathtt{push}((f,Y) \blacktriangleright \sigma')$, z^u of u and z_i of B_i for all i. By construction of $\mathcal{C}_\mathcal{G}$, $(A,X,\mathtt{push}((f,Y) \blacktriangleright \sigma')) \to (B,Y,\sigma')$ must be a rule of $\mathcal{C}_\mathcal{G}$. Hence, by induction hypothesis, $(A,X)[z''](\to_{pump} + \to_{skip})^*(A,X)[(f,Y)z']$ with z' an expansion of σ'. As a result, we have $(A,X)[\overline{z}_1] = (A,X)[z''z^u z_1 \dots z_k](\to_{pump} + \to_{skip})^*(A,X)[(f,Y)z'z^u z_1 \dots z_k]$. Thus $z'z^u z_1 \dots z_k$ is an expansion of $\sigma' u B_1 \dots B_k = \sigma_2$.

b. Otherwise, $\mathbf{height}((f,Y)\sigma') = h$ and $(f,Y)\sigma'$ is an h-atom.

i. If $((f,Y)\sigma')u$ is of the form $v_0v_1\dots v_Nw$ with $\varphi(v_i) = e$ for all i, for some $e \in \mathbf{Idem}(\mathbb{S})$, we have two cases.

A. If there exists j such that B_j is of the form $e^+v'_1\dots v'_Nw'$ and $\varphi(v_1\dots v_NB_1\dots B_{j-1}e) = e$ then we have $\sigma_1 = (e^+v'_1\dots v'_Nw')B_{j+1}\dots B_k$. In that case, $\overline{z}_1 = z'_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k$ where $z_j^{v'}$ is an expansion of v'_j for all j, z'_e is an expansion of e^+ and $z^{w'}$ of w'. Let z_e be an expansion of $v_0v_1\dots v_NB_1\dots B_{j-1}e^+$.
We can use a pump rule and a skip rule:

$$(A,X)[\overline{z}_1] = (A,X)[z'_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k]$$
$$\to_{pump} (A,X)[z_1^{v'}\dots z_N^{v'}z'_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k]$$
$$\to_{skip} (A,X)[z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k]$$
$$\to_{pump} (A,X)[z_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k]$$

Since z_e is an expansion of $v_0\dots v_NB_1\dots B_{j-1}$, $z_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k$ is an expansion of $v_0\dots v_NB_1\dots B_{j-1}e^+v'_1\dots v'_Nw'B_{j+1}\dots B_k = ((f,Y)\sigma')uB_1\dots B_k = (f,Y)\sigma_2$. Hence $z_e z_1^{v'}\dots z_N^{v'}z^{w'}z_{j+1}\dots z_k$ is of the form $(f,Y)\overline{z}_2$ with $\overline{z}_2$ an expansion of σ_2.

B. Otherwise $\sigma_1 = (e^+v_1\dots v_Nw)B_1\dots B_k$. Hence $\overline{z}_1 = z_e z_1^v\dots z_N^v z^w z_1\dots z_k$ where z_j^v is an expansion of v_j for all j, z_e is an expansion of e^+ and z^w of w.
Let z_0^v be an expansion of v_0. We have $(A,X)[\overline{z}_1] = (A,X)[z_e z_1^v\dots z_N^v z_1\dots z_k] \to_{skip} (A,X)[z_1^v\dots z_N^v z_1\dots z_k] \to_{pump} (A,X)[z_0^v z_1^v\dots z_N^v z_1\dots z_k]$. The resulting stack content $z_0^v\dots z_N^v z_1\dots z_k$ is an expansion of $(f,X)\sigma_2$. Hence it is of the form $(f,Y)\overline{z}_2$ with $\overline{z}_2$ an expansion of σ_2.

ii. Otherwise, we have $\sigma_1 = ((f,Y)\sigma')uB_1\dots B_k$ and thus $\overline{z}_1 = (f,Y)\overline{z}_2$ with $\overline{z}_2$ a expansion of σ_2.

◀

H Proof of Proposition 6.2

We prove the following statement.

▶ **Proposition 6.2.** $L^{pump,skip}(\overline{\mathcal{G}}) \subseteq L(\overline{\mathcal{G}})\downarrow$

▶ **Lemma H.1.** *Let $(A,X)[\overline{z}]$ be a term of $\overline{\mathcal{G}}$, $e \in \mathbf{Idem}(\mathbb{S})$ and $z_e \in \overline{I}^*$ such that $\varphi(z_e) = e$. Then $L((A,X)[z_e\overline{z}]) \subseteq L((A,X)[\overline{z}])\downarrow$.*

Proof. Let $w \in L((A,X)[\overline{z}])$. There is a derivation $(A,X)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$. Since $\varphi(z_e) = e$, by Lemma 5.1, z_e is feasible, i.e., there is a derivation $(A,X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(B,X)[z_e]v$ with $u,v \in \mathbf{SF}$. Furthermore, since e is idempotent, by definition of the product of $\mathbb{S}$, we have $B\ \mathcal{R}_X\ A$, i.e., there is a derivation $(B,X) \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u'(A,X)v'$.

In total, we get $(A,X)[\overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} u(B,X)[z_e]v \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} uu'(A,X)[z_e\overline{z}]v'v \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} uu'wv'v$. ◀

▶ **Lemma H.2.** *Let $(A,X)[v_1\dots v_Nz_ev_1\dots v_N\overline{z}]$ be a term of $\overline{\mathcal{G}}$, and $e = (B,X,M,A,X) \in \mathbf{Idem}(\mathbb{S})$ such that $\varphi(v_1) = \dots = \varphi(v_N) = \varphi(z_e) = e$.*
Then $L((A,X)[v_1\dots v_N\overline{z}]) \subseteq L((A,X)[v_1\dots v_Nz_ev_1\dots v_N\overline{z}])\downarrow$.

Proof. Let $w \in L((A,X)[u_1 \dots u_N \overline{z}])$. Let τ be a derivation tree for $(A,X)[u_1 \dots u_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w$.

Consider V the set of nodes ν such that:

1. Either ν is a leaf with a label in T^* and $\overline{z}$ is a suffix of the stack content of all its strict ancestors,
2. or ν is labeled $(C,Y)[v_i \dots v_N \overline{z}]$ for some $i \in \{1, \dots, N+1\}$ and $v_i \dots v_N \overline{z}$ is a suffix of the stack content of all its ancestors. If $M(B,B) = \top$ we say that ν is a *special node*.

▷ Claim H.3. Every branch of τ contains either a node of the first type or a special node.

Proof. Consider a branch of τ. If the leaf of this branch is not of the first type, then the prefix $v_1 \dots v_N$ must be fully popped.

Therefore, there are nodes $\nu_1, \dots, \nu_{N+1}$ of the second type along this branch: each ν_i is labeled $(A_i, X_i)[v_i \dots v_N \overline{z}]$ and all its ancestors have a stack content with suffix $v_i \dots v_N \overline{z}$.

In consequence, for all i we can define the derivation tree obtained by restricting τ to descendants of ν_i, and then removing all nodes where $v_{i+1} \dots v_N \overline{z}$ is not a suffix of the stack and their descendants, as well as strict descendants of ν_{i+1}. By definition of ν_{i+1}, it is a leaf of the resulting tree.

We obtain a derivation $(A_i, X_i)[v_i \dots v_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_i (A_{i+1}, X_{i+1})[v_{i+1} \dots v_N \overline{z}] w_i'$ with $w_i, w_i' \in \mathbf{SF}$. By definition of M, since $\varphi(v_i) = e$, we have $X_i = X_{i+1} = X$ and $M(A_i, A_{i+1}) = \top$.

By Lemma 6.1, since $M(A_i, A_{i+1}) = \top$ for all i, there exists i such that $M(A_i, A_i) = \top$. As a consequence, ν_i is a special node. ◁

Let ν be a special node, labeled $(C,X)[u_i \dots u_N \overline{z}]$ with $M(C,C) = \top$. Since $\varphi(v_1) = \dots = \varphi(v_N) = \varphi(z_e) = e$, $\varphi(v_i \dots v_N z_e v_1 \dots v_{i-1}) = e$. As a consequence, since $M(C,C) = \top$, we have a derivation $(C,X)[v_i \dots v_N z_e v_1 \dots v_{i-1}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_-(C,X)w_+$ with $w_-, w_+ \in \mathbf{SF}$.

Consider the set of minimal nodes for the ancestor relation, which are either of the first type or special. They form a maximal antichain for the ancestor relation, and intersect every branch.

We can thus define the subtree whose root is the same as τ and whose leaves are those nodes. By definition, every node in this tree is labeled with a term whose stack has $\overline{z}$ as suffix. Define τ' the tree obtained by replacing each suffix $\overline{z}$ with $z_e u_1 \dots u_N \overline{z}$. The resulting tree τ' is still a valid derivation tree.

Below each leaf ν of τ' which is a special node we append with a label of the form $(C,X)[v_i \dots v_N \overline{z}]$, we append a derivation tree for

$$(C,X)[v_i \dots v_N z_e v_1 \dots v_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_-(C,X)[v_i \dots v_N \overline{z}]w_+ \text{ with } w_-, w_+ \in \mathbf{SF}$$

. Then we append the subtree of τ rooted in ν at the resulting leaf labeled $(C,X)[v_i \dots v_N \overline{z}]$.

We obtain a derivation tree from $(A,X)[z' v_1 \dots v_N]$ to a sentential form $\overline{w}$ of which w is a subword. Since $\overline{\mathcal{G}}$ is sound and $(A,X)[z' v_1 \dots v_N]$ is reachable, there is a word $w' \in T^*$ such that $\overline{w} \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w'$. Since $w \preceq \overline{w}$, $w \preceq w'$, proving the result. ◀

Proof of Proposition 6.2. We show that for all sentential form u of $\overline{\mathcal{G}}$, $L(u)^{pump,skip} \subseteq L(u)\downarrow$.

Let $w \in L(u)^{pump,skip}$, we proceed by induction on the derivation $u \overset{*}{\Rightarrow}^{pump,skip}_{\overline{\mathcal{G}}} w$, and distinguish cases according to the first step.

- If the first step is $u \Rightarrow_{\overline{\mathcal{G}}} u'$ then $u' \overset{*}{\Rightarrow}^{pump,skip}_{\overline{\mathcal{G}}} w$. By induction hypothesis, $u' \Rightarrow_{\overline{\mathcal{G}}} w'$ with $w \preceq w'$, hence $u \Rightarrow_{\overline{\mathcal{G}}} u' \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w'$.

- If the first step is $u = u_1(A,X)[z_e\overline{z}]u_2 \to_{pump} u_1(A,X)[z'_e z_e \overline{z}]u_2 = u'$ then $u' \overset{*}{\Rightarrow} \overset{pump,skip}{\overline{\mathcal{G}}} w$. By induction hypothesis, $u' \Rightarrow_{\overline{\mathcal{G}}} w'$ with $w \preceq w'$. Let $w' = w_1 w_A w_2$ such that $u_1 \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_1$, $u_2 \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_2$ and $(A,X)[z'_e z_e \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_A$. By Lemma H.1, $u \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_1 w'_A w_2$ with $w_A \preceq w'_A$. We obtain the result since $w \preceq w' \preceq w_1 w'_A w_2$.
- If the first step is $u = u_-(A,X)[z_e v_1 \dots v_N \overline{z}]u_+ \to_{skip} u_-(A,X)[v_1 \dots v_N \overline{z}]u_+ = u'$ then $u' \overset{*}{\Rightarrow} \overset{pump,skip}{\overline{\mathcal{G}}} w$. By induction hypothesis, $u' \Rightarrow_{\overline{\mathcal{G}}} w'$ with $w \preceq w'$. Let $w' = w_- w_A w_+$ such that $u_- \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_-$, $u_+ \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_+$ and $(A,X)[v_1 \dots v_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_A$. By Lemma H.1, $(A,X)[v_1 \dots v_N z_e v_1 \dots v_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w'_A$ with $w_A \preceq w'_A$. By Lemma H.2, $(A,X)[z_e v_1 \dots v_N \overline{z}] \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w''_A$ with $w'_A \preceq w''_A$.
  We thus have $u \overset{*}{\Rightarrow} u_- w''_a u_+$. Further, $u_- w''_A u_+ \overset{*}{\Rightarrow}_{\overline{\mathcal{G}}} w_- w''_A w_+$. We obtain the result since $w \preceq w' \preceq w_- w'_A w_+ \preceq w_- w''_A w_+$.

The induction is proven. To obtain the lemma, it suffices to apply it to $u = (S, \mathbf{U})$. ◀

I Additional material from Section 9

▶ **Lemma I.1.** *There is an execution of $\mathcal{G}_k$ that produces the word $\mathtt{a}^{\exp_3(k)}$*

Proof. Let $\alpha_1 \cdots \alpha_m \in \Sigma_k^*$ be the unique word accepted by all $\mathcal{A}_i$, with $m = 2^k$. For all $b_1, \dots, b_m \in \{0,1\}$, we write $\overline{b_1 \cdots b_m}$ for the number in $[0, 2^m - 1]$ whose binary representation over m bits is $b_1 \cdots b_m$, b_1 being the bit of smallest weight.

We show that for all $b_1, \dots, b_m \in \{\mathtt{0},\mathtt{1}\}$, if $M = \overline{b_1 \cdots b_m}$ then $Z(\alpha_1, b_1) \cdots (\alpha_m, b_m)\bot$ produces $\mathtt{a}^{2^{2^m - M}}$, by induction on $2^m - M$.

We can apply $Z \xrightarrow{\mathcal{B}_1, \dots, \mathcal{B}_k} D$ and $D \to AA$ to obtain two copies of $A(\alpha_1, b_1) \cdots (\alpha_m, b_m) f_\bot$. This is because $\alpha_1 \cdots \alpha_m$ is accepted by all $\mathcal{A}_i$. We are left with

$$A[(\alpha_1, b_1) \cdots (\alpha_m, b_m)\bot]A[(\alpha_1, b_1) \cdots (\alpha_m, b_m)\bot].$$

If $b_i = \mathtt{1}$ for all i, then we can apply $A(\alpha_i, \mathtt{1}) \to A$ for each i and then $A\bot \to F$ and $F \to \mathtt{a}$ to obtain $\mathtt{aa}$, which is what we want since we would then have $M = 2^m - 1$ and thus $\mathtt{a}^{2^{2^m - M}} = \mathtt{a}^2$.

Otherwise, let j be the least index such that $b_j = \mathtt{0}$. It suffices to show that $A(\alpha_1, b_1) \cdots (\alpha_m, b_m)\bot$ produces $\mathtt{a}^{2^{2^m - M - 1}}$. We have $M + 1 = \overline{\mathtt{1}^{j-1}\mathtt{0} b_{j+1} \cdots b_m} + 1 = \overline{\mathtt{0}^{j-1}\mathtt{1} b_{j+1} \cdots b_m}$.

We can apply:

- $A(\alpha_i, \mathtt{1}) \to A$ for each $i < j$ until we get $A(\alpha_j, \mathtt{0})(\alpha_{j+1}, b_{j+1}) \cdots (\alpha_m, b_m)\bot$,
- then apply $A(\alpha_j, \mathtt{0}) \to B$ and $B \to Z(\alpha_j, \mathtt{1})$ to get $Z(\alpha_j, \mathtt{1})(\alpha_{j+1}, b_{j+1}) \cdots (\alpha_m, b_m)\bot$,
- and $Z \to Z(\alpha_i, \mathtt{0})$ for each $i < j$, in decreasing order, until we obtain $Z(\alpha_1, \mathtt{0}) \cdots (\alpha_{j-1}, \mathtt{0})(\alpha_j, \mathtt{1})(\alpha_{j+1}, b_{j+1}) \cdots (\alpha_m, b_m)\bot$, which produces $\mathtt{a}^{2^{2^m - M - 1}}$ by induction hypothesis.

The induction is proven. To obtain the lemma, it suffices to start with S, apply $S \to Z\bot$ and then $Z \to Z(\alpha_i, \mathtt{0})$ for each $i \in [1, k]$ in decreasing order. We get $Z(\alpha_1, \mathtt{0}) \cdots (\alpha_k, \mathtt{0})$, which produces $a^{\exp_3(k)}$ by applying the induction with $M = 0$. ◀

▶ **Lemma I.2.** *For all $E \in N_k$ and $z \in I_k^*$ the language $L_\emptyset(Ez)$ contains at most one word. In particular, $L(\mathcal{G}_k)$ is empty or a singleton.*

Proof. We prove this for each $E \in N_k$, one by one.

1. We first observe that from a configuration Zz there is always at most one rule which can lead to a complete derivation tree: if z does not contain $\bot$ then $\mathsf{L}_\emptyset(Zz) = \emptyset$. If $z = (\alpha_1, b_1) \cdots (\alpha_n, b_n) \bot z'$ then:
   - if $n \neq 2^k$ we cannot apply $Z \xrightarrow{\mathcal{B}_1,\dots,\mathcal{B}_k} D$ since z has no prefix accepted by all $\mathcal{B}_i$. Furthermore, if $n > 2^k$, we have $\mathsf{L}_\emptyset(Zz) = \emptyset$ since we can only push more pairs $(\alpha, 0)$ on the stack, so we will never be able to apply $Z \xrightarrow{\mathcal{B}_1,\dots,\mathcal{B}_k} D$.
   - if $n = 2^k$ then we cannot apply $Z \to Z(\alpha, 0)$, by the previous item, as we would obtain more than 2^k symbols before the first $\bot$.
2. Note that B, D and S all have a single rule. Meanwhile, A has several but the top stack symbol determines which rule can be applied. In conclusion, from every configuration Ez, there is at most one rule that can be applied to lead to a complete derivation. As a consequence, the language of $\mathcal{G}$ contains at most one word.

◀

By combining the two previous statements we conclude that $\mathcal{G}_k$ recognises the singleton language $\{a^{\exp_3(k)}\}$, while having size only quadratic in k. A trim NFA for this language must be acyclic, as otherwise it would recognise an infinite language, and thus have at least $\exp_3(k)$ states.